



B.N.M. Institute of Technology

An Autonomous Institution under VTU

**PROJECT WORK: PHASE II (21ECP183)
REPORT**

ON

**“DESIGN AND VERIFICATION OF SYNTHESIZABLE LOW POWER 8-BIT
RISC MICROCONTROLLER.”**

Submitted in partial fulfillment of the requirements for the award of degree of

**BACHELOR OF ENGINEERING
in**

ELECTRONICS AND COMMUNICATION ENGINEERING

by

Sanjana H S

1BG21EC080

Thanishka K

1BG21EC098

Ujwal B S

1BG21EC101

Under the guidance of

Internal Guide

Dr. Yasha Jyothi M Shirur

Professor and Head

Dept. of ECE

BNMIT, Bangalore

External Guide

Dr. Manjunatha V

Principal Engineer

Synopsys

Department of Electronics & Communication Engineering

2024 – 2025

B.N.M. Institute of Technology

An Autonomous Institution under VTU, Approved by AICTE, Accredited as 'A' Grade Institution by NAAC till 31.12.2026

All Eligible UG branches – CSE, ECE, EEE, ISE & Mech.Engg. Accredited by NBA till 30.06.2025

Post box no. 7087, 27th cross, 12th Main, Banashankari 2nd Stage, Bengaluru- 560070, INDIA

Ph: 91-80- 26711780/81/82 Email: principal@bnmit.in, www.bnmit.org

Department of Electronics and Communication Engineering



CERTIFICATE

Certified that the project work entitled “**Design and Verification of Synthesizable Low Power 8-Bit RISC Microcontroller**” carried out by **Sanjana H S (1BG21EC080)**, **Thanishka K (1BG21EC098)** and **Ujwal B S (1BG21EC101)** bona fide students of **VIII semester** in partial fulfillment for the award of **Bachelor of Engineering** degree in **Electronics and Communication Engineering** of the Visvesvaraya Technological University, Belagavi during the year **2024-2025**.

It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the department library. The Project Work Phase II report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said degree.

**Dr. Yasha Jyothi M
Shirur**

Professor and Head
Dept. of ECE
BNMIT, Bangalore

Dr. Yasha Jyothi M Shirur

Professor and Head
Dept. of ECE
BNMIT, Bangalore

Dr. S Y Kulkarni
Additional Director &
Principal
BNMIT
Bangalore

	Internal Examiner	External Examiner
Name		
Signature		

B.N.M. Institute of Technology

An Autonomous Institution under VTU, Approved by AICTE, Accredited as 'A' Grade Institution by NAAC till 31.12.2026

All Eligible UG branches – CSE, ECE, EEE, ISE & Mech.Engg. Accredited by NBA till 30.06.2025

Post box no. 7087, 27th cross, 12th Main, Banashankari 2nd Stage, Bengaluru- 560070, INDIA

Ph: 91-80- 26711780/81/82 Email: principal@bnmit.in, www.bnmit.org

Department of Electronics and Communication Engineering



DECLARATION

We, **Sanjana H S (1BG21EC080)**, **Thanishka K (1BG21EC098)** and **Ujwal B S (1BG21EC101)** hereby declare that the entire work submitted in this Project report with the title “**Design and Verification of Synthesizable Low Power 8-Bit RISC Microcontroller.**” has been carried out by us at the Department of Electronics and Communication Engineering, BNM Institute of Technology, Bengaluru, under the supervision of **Dr. Yasha Jyothi M Shirur**. The work submitted is our original work and has not been submitted by us anywhere for the award of any other degree in this university or any other university.

Sanjana H S 1BG21EC080

Thanishka K 1BG21EC098

Ujwal B S 1BG21EC101

Abstract

In the era of portable electronics and embedded systems, the demand for energy-efficient microcontrollers has become increasingly critical to sustain the functionality of battery-operated devices. This project presents the design and implementation of a low-power 8-bit Reduced Instruction Set Computer (RISC) microcontroller specifically tailored for compact electronic applications, such as Internet of Things (IoT) devices and wearable technologies.

The RISC architecture is chosen for its inherent simplicity and efficiency, which facilitate rapid instruction execution with minimal cycles. This design philosophy not only optimizes performance but also reduces the complexity of the control logic, allowing for faster processing speeds and improved overall system responsiveness. The microcontroller features a compact 4-bit opcode instruction set, which enables efficient encoding of operations while minimizing the resource footprint.

Key design elements include a streamlined control unit that enhances throughput and minimizes delays during instruction execution. The architecture is further augmented by the integration of advanced power management techniques, including clock gating, power gating, and dynamic voltage scaling. Clock gating selectively disables the clock signal to inactive components, thereby reducing dynamic power consumption. Power gating completely disconnects power from non-active units, effectively minimizing leakage current. Dynamic voltage scaling adjusts the supply voltage based on workload demands, optimizing energy usage during varying operational states.

This microcontroller strikes a critical balance between processing power and energy efficiency, thereby extending battery life and enhancing reliability in applications where power availability is limited. Designed with scalability the architecture provides a robust foundation for future enhancements, including the potential development of 16-bit or 32-bit versions to meet evolving technological demands.

The design and simulation processes were carried out using Cadence Genus for synthesis and verification, Xilinx ISE (version 14.2) for simulation ensuring a comprehensive approach to the development of microcontroller. This project contributes to the advancement of sustainable, energy-efficient microcontrollers. It addresses the need for reliable processing in modern electronics, paving the way for the next generation of portable and embedded systems that prioritize both performance and energy conservation.

Acknowledgement

We would like to place on record our sincere thanks and gratitude to the concerned people, whose suggestions and words of encouragement has been valuable.

We express our heartfelt gratitude to **BNM Institute of Technology**, for giving us the opportunity to pursue Degree of Electronics and Communication Engineering and helping us to shape our career. We take this opportunity to thank **Prof. T. J. Rama Murthy**, Director, **Dr. S.Y. Kulkarni**, Additional Director and Principal, **Prof. Eishwar N Maanay**, Dean and **Dr. Krishnamurthy G.N.**, Deputy Director for their support and encouragement to pursue this project. We would like to thank **Dr. Yasha Jyothi M Shirur**, Professor and Head, Dept. of Electronics and Communication Engineering, for her support and encouragement.

We would like to thank our guide **Dr. Yasha Jyothi M Shirur**, Professor & HOD, Dept. of Electronics and Communication Engineering, who has been the source of inspiration throughout our project work and has provided us with useful information at every stage of our project.

We would also like to thank our project coordinator **Dr. Ashwini S Savanth**, Associate Professor, Dept. of Electronics and Communication Engineering, who is kind enough to extend her help for our project whenever the need arose.

I express my hearty thanks to our External Guide at **Synopsys**, **Dr.Manjunatha V**, for his guidance and unconditional support, despite of his busy work schedule.

Finally, we are thankful to all the teaching and non-teaching staff of Department of Electronics and Communication Engineering for their help in the successful completion of our project. Last but not the least we would like to extend our sincere gratitude to our parents and all our friends who were a constant source of inspiration.

Sanjana H S 1BG21EC080

Thanishka K 1BG21EC098

Ujwal B S 1BG21EC101

Table of Contents

Abstract.....	i
Acknowledgement.....	ii
Table of Contents.....	iii
List of Figures.....	iv
List of Tables.....	vi
Chapter 1- Introduction.....	1
1.1 Motivation for the Project Work	
1.2 Problem Statement	
1.3 Objectives of the Project Work	
1.4 Organization of Project Report	
Chapter 2-Literature Survey.....	6
2.1 Overview of RISC Microprocessor Design	
2.2 Optimization Techniques	
2.3 Design Methodologies	
2.4 Verification Techniques	
Chapter 3 – Design Specification and Methodology	13
3.1 Software Requirements	
3.2 Methodology	
2.3 Design Methodologies	
2.4 Verification Techniques	
Chapter 4 -Low Level Processor Design Module Implementation	21
4.1 Microprocessor Components	
4.2 Low Power Techniques	
Chapter 5 – Results	39
Chapter 6 – Conclusion	53
6.1 Conclusion	
References	
Appendix	

List of Figures

Figure No.	Description	Page No
1.1	Diagram of RISC Architecture	2
3.1	Flow Diagram of RISC Microcontroller	19
4.1	Arithmetic Logic Unit Schematic	22
4.2	Arithmetic Logic Unit Waveform	22
4.3	Program Counter Schematic	23
4.4	Program Counter Waveform	23
4.5	Control Unit Schematic	24
4.6	Control Unit Waveform	25
4.7	Memory Address Register Schematic	26
4.8	Memory Address Register Waveform	26
4.9	Static Random-Access Memory Schematic	27
4.10	Static Random-Access Memory Waveform	27
4.11	Instruction Register Schematic	28
4.12	Instruction Register Waveform	29
4.13	Ring Counter Schematic	30
4.14	Ring Counter Waveform	30
4.15	Accumulator Schematic	31
4.16	Accumulator Waveform	31
4.17	Output Register Schematic	32
4.18	Output Register Waveform	32
4.19	Instruction Decoder Schematic	33
4.20	Instruction Decoder Waveform	34
4.21	8-bit Microcontroller with Clock Gating Schematic before synthesis	35
4.22	8-bit Microcontroller with Power Gating Schematic before synthesis	37
4.23	8-bit Microcontroller with Dynamic Voltage Scaling Schematic before synthesis	38
5.1	8-bit Microcontroller Waveform Callout	40
5.2	8-bit Microcontroller Waveform	41

Figure No.	Description	Page No
5.3	8-bit Microcontroller Schematic	42
5.4	8-bit Microcontroller Schematic after Synthesis	43
5.5	8-bit Microcontroller Power reports from Cadence	43
5.6	8-bit Microcontroller with Clock Gating Power reports from Cadence	44
5.7	8-bit Microcontroller with Clock Gating Schematic after synthesis	44
5.8	8-bit Microcontroller with Power Gating Power reports from Cadence	45
5.9	8-bit Microcontroller with Power Gating Schematic after synthesis	46
5.10	Waveform of the power gated modules	46
5.11	8-bit Microcontroller with Dynamic Voltage Scaling Schematic after synthesis	47
5.12	8-bit Microcontroller with Dynamic Voltage Scaling Power reports from Cadence	47
5.13	Waveform of the Dynamic Voltage Scaling modules	48
5.14	8-bit Microcontroller with Clock gating and Power gating low power techniques applied Schematic after synthesis	48
5.15	8-bit Microcontroller Clock gating and Power gating low power techniques applied Power reports from Cadence	49
5.16	8-bit Microcontroller with all three low power techniques applied Schematic after synthesis	49
5.17	8-bit Microcontroller all three low power techniques applied Power reports from Cadence	50
5.18	Comparison of Power Consumption Across Different Low-Power Techniques	51

List of Tables

Table No.	Description	Page No
1.1	Basic features of popular microprocessors	3
2.1	Optimization Techniques	9
3.1	Design Specifications of RISC Microcontroller	14
3.2	Instruction Set	15-16
5.1	Comparison Table of the 8-bit RISC Microcontroller Designs	51

CHAPTER 1

INTRODUCTION

Chapter 1

INTRODUCTION

1. Introduction

In the era of embedded systems and portable electronics, power efficiency is vital in microcontroller design. This project focuses on developing a low-power 8-bit RISC microcontroller optimized for small devices, emphasizing minimal lag and improved efficiency. The RISC architecture is chosen for its simplicity, enabling faster execution and efficient instruction handling with a compact instruction set and small opcode width.

To further enhance energy efficiency, the design incorporates power-saving techniques such as clock gating, power gating, and dynamic voltage scaling, which reduce overall power consumption by selectively disabling unused components and adjusting voltage levels based on processing needs. By integrating these low-power strategies with RISC architecture, the microcontroller achieves an optimal balance between performance and energy efficiency, making it well-suited for modern low-power applications.

The combination of RISC architecture and these low-power techniques enables the microcontroller to achieve an optimal balance between performance and energy efficiency. This makes it particularly suitable for modern low-power applications, such as Internet of Things (IoT) devices, wearable technology, and other embedded systems that require reliable performance without compromising battery life.

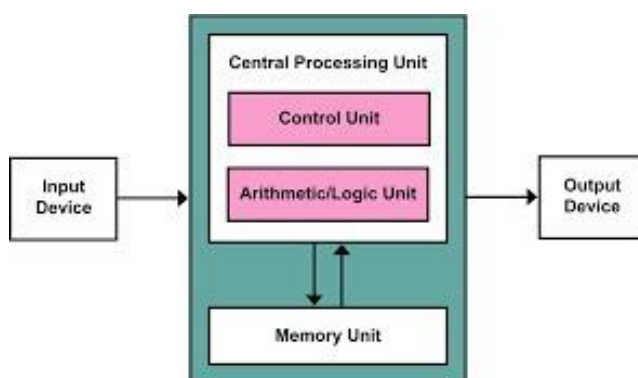


Figure 1.1:- Diagram of RISC Architecture

Moreover, the design is scalable and adaptable, providing a foundation for future enhancements, such as the development of 16-bit or 32-bit versions. This scalability ensures that the microcontroller can meet the evolving demands of technology while maintaining its focus on energy efficiency.

By addressing the current needs for low-power processing, this project contributes to the advancement of sustainable microcontroller technology, supporting the growing trend towards energy-efficient solutions in contemporary electronics.

1.1 Motivation for the Project Work

In today's rapidly evolving technological landscape, portable electronics, IoT devices, and embedded systems are crying out for smarter, more energy-efficient processors. The growing demand for devices with extended battery life, compact designs, and reliable performance has sparked the development of an innovative 8-bit RISC microcontroller.

Table 1.1: - Basic features of popular microprocessors

Microprocessor	Year	Clock Speed (GHz)	Caches (MB)
Intel 486 (Intel, Santa Clara, CA, USA)	1989	25 to 50	8
Intel Pentium Pro (Intel, Santa Clara, CA, USA)	1995	200	8/8
DEC Alpha 21164 (DEC, Maynard, MA, USA)	1996	500	8/8/96
Power PC 604e	1997	233	32/32
AMD K5 (AMD, Santa Clara, CA, USA)	1996	75-133	8/16
MIPS R10000 (MIPS Technologies, Sunnyvale, CA, USA)	1996	200	32/32
Intel Pentium IV (Intel, Santa Clara, CA, USA)	2000	1400-2000	256
AMD Athlon 64 X2	2005	2	0.5
Intel Core 2 Duo	2006	2.66	4
Intel Core 2 Quad Q6600	2007	2.4	8
Intel Core i7-3770	2012	3.4	8
AMD Ryzen 7 1700x	2017	3-3.6	4/16
Intel Core i9 10900	2020	5.20	20

By embracing the RISC architecture's inherent simplicity, this project aims to create a controller that can execute tasks quickly and efficiently, consuming minimal power - a critical requirement for modern battery-operated systems.

The proposed microcontroller goes beyond basic design principles by incorporating advanced power management techniques like clock gating, power gating, and dynamic voltage scaling. These strategies ensure optimal performance across varying workloads while maintaining energy efficiency. Moreover, the design's scalability positions it as a forward-looking solution, capable of adapting to future technological needs. This project not only addresses current low-power computing challenges but also anticipates the evolving requirements of next-generation portable and embedded electronics.

1.2 Problem Statement

Traditional microcontroller designs often struggle to meet the stringent power consumption requirements of these applications, leading to reduced battery life and increased operational costs. This project aims to address the challenge of developing an 8-bit Reduced Instruction Set Computer (RISC) microcontroller that is specifically optimized for low-power operation. The RISC architecture is chosen for its streamlined instruction set, which allows for faster execution and reduced complexity in instruction handling. However, the inherent simplicity of RISC does not automatically guarantee low power consumption; thus, it is essential to integrate advanced power management techniques into the design.

1.3 Objectives of the Project Work

The primary objective of this project is to design a low-power 8-bit RISC microcontroller optimized for compact electronic applications by incorporating power-saving techniques such as clock gating power gating, Multi voltage threshold, etc. The processor is aimed at striking a balance between processing power and energy efficiency, with a focus on embedded and portable devices. Key goals include efficient instruction handling with minimal cycles and a small resource footprint, enhancing scalability, cost-efficiency, and suitability for a range of energy-sensitive applications.

The scope encompasses the development of an 8-bit microcontroller with a configurable pipeline structure and simplified control logic tailored for battery-operated and portable devices. Suitable for IoT applications, wearable technology, and other embedded systems. The result is a processor well suited for modern applications that prioritize low power consumption and compact form factors.

1.4 Organization of Project Report

Chapter 1: Introduction

This chapter provides an overview of the project, including the motivation behind the development of a low-power 8-bit RISC microcontroller, the problem statement, and the objectives of the project work.

Chapter 2: Literature Survey

This chapter reviews existing research and developments in the field of low-power microcontroller, RISC architecture, and relevant power management techniques.

It highlights the current state of technology and identifies gaps that the project aims to address.

Chapter 3: Methodology

This chapter outlines the design and development process of the microcontroller, detailing the methodologies employed in the implementation of the RISC architecture, power management techniques, and the overall design framework.

Chapter 4: Low Level Processor Design Module Implementation

This chapter describes the actual implementation of the 8-bit RISC microcontroller, including hardware and software components, design tools used, and the integration of power management features.

Chapter 5: Results

This chapter presents the results obtained from the simulation and testing of the low-power 8-bit RISC microcontroller designed in this project. The results are derived from various design tools, including Xilinx ISE and Cadence Genus, which were utilized for synthesis, simulation, and performance evaluation. The focus is on the performance metrics, power consumption, and overall functionality of the microcontroller.

Chapter 6: Conclusion

This chapter summarizes the findings of the project, reflecting on the achievements and challenges encountered. It also outlines potential future enhancements and applications of the microcontroller design, including scalability to 16-bit or 32-bit architectures.

CHAPTER 2

LITERATURE SURVEY

Chapter 2

LITERATURE SURVEY

2.1 Overview of RISC Microcontroller Design

The demand for low-power microcontroller has surged with the proliferation of mobile devices, IoT applications, and embedded systems. As these applications often rely on battery power, optimizing energy consumption while maintaining performance has become a critical design goal [1-3]. Low power design techniques are essential to extend battery life and reduce thermal output, making them a focal point in modern microcontroller development. The RISC (Reduced Instruction Set Computing) architecture, known for its simplicity and efficiency, is particularly well-suited for low-power applications due to its streamlined instruction set and reduced complexity in control logic [4],[5]. The 8-bit RISC microcontroller, while limited in data processing capabilities, offers a compelling solution for simple tasks that require minimal power consumption [6].

RISC architectures are characterized by their use of a small, highly optimized instruction set that allows for faster execution of instructions. This design philosophy leads to a more efficient use of resources, which is crucial for low-power applications [7]. The fundamental principles of RISC include a large number of registers, a load/store architecture, and a focus on pipelining to improve instruction throughput. These features enable RISC microcontrollers to execute instructions in fewer cycles compared to their Complex Instruction Set Computing (CISC) counterparts, resulting in lower energy consumption per instruction [8].

In the context of 8-bit microcontrollers, the design must also consider the limitations imposed by the 8-bit data bus and addressable memory space. While these constraints can limit performance, they also drive innovation in design techniques aimed at maximizing efficiency. Various architectural strategies, such as optimizing the instruction set for common operations and employing efficient data handling techniques, have been explored in recent literature [9]. Furthermore, the integration of low power techniques at the architectural level, such as adaptive voltage scaling and clock gating, has been shown to significantly enhance the energy efficiency of 8-bit RISC microcontrollers [10].

2.2 Optimization Techniques

A comparative summary of key power management techniques in microcontroller design is presented in Table 2, outlining their power savings potential, ideal applications, key advantages, and limitations

1.Clock Gating Techniques

Clock gating is among the most widely adopted methods for reducing dynamic power consumption. By selectively turning off clock signals in inactive parts of the processor, significant power savings were achieved across designs [11], [12]. Combinational and sequential clock gating methods were reported to reduce dynamic power by up to 20-25%, with higher gains in designs that implemented flip-flop or multiplexer-based gating. In an FPGA-based RISC processor implementation, clock gating reduced dynamic power by approximately 15% without significant impact on processing speed [13]. Advanced versions of clock gating, such as deterministic clock gating, predicted circuit activity cycles ahead to maximize idle time, yielding up to 19.9% power savings.

2.Power Gating and Leakage Reduction

Power gating further minimizes power usage by disconnecting the power supply to inactive units, thus reducing leakage current. Studies on 8-bit RISC processors found that combining power and clock gating reduces both dynamic and leakage power effectively [15]. Using power gating alone yielded an approximate 20% reduction in leakage power in CMOS circuits, particularly during idle states [16]. Adaptive power gating approaches allowed processors to enter low-power states dynamically, minimizing standby power without affecting performance during active states. Notably, an automated implementation of power gating in VHDL reduced leakage power on an FPGA by approximately 16% [17].

3.Pipeline Optimization

Pipelining enhances the efficiency of instruction execution, enabling faster processing without increasing power usage [18]. By distributing tasks across multiple stages (e.g., fetch, decode, execute, write-back), pipelining reduces the clock cycles needed per instruction. Studies found that, in pipelined RISC processors, clock gating each pipeline stage individually further reduced power consumption by minimizing clock signal propagation [19]. Pipeline optimization strategies also showed that parallel instruction processing, combined with selective gating, could achieve substantial power savings, particularly in applications where high instruction throughput is required [20].

4.Voltage and Frequency Scaling

Dynamic voltage and frequency scaling (DVFS) emerged as a critical technique, with studies showing that adjusting the processor's operating voltage and frequency to workload demands can lower power usage significantly [21]. Voltage scaling alone accounted for up to 25% power reduction, while combined voltage-frequency scaling (DVFS) saved an additional 15% when applied based on real-time workload analysis [22]. By dynamically adjusting frequency according to task requirements, these designs achieved energy efficiency without sacrificing performance. However, studies noted that DVFS requires careful management to avoid latency, making it ideal for applications with variable processing demands, such as IoT devices [23].

5.Multi-Threshold CMOS (MTCMOS)

MTCMOS technology employs transistors with different threshold voltages to optimize performance in active areas while controlling leakage in inactive areas. This approach allows for high performance in critical sections and power savings in non-critical sections by using high-threshold transistors that minimize leakage [24].

Table 2.1: - Optimization Techniques

Power Management Technique	Power Savings (%)	Ideal Application	Key Advantages	Limitations
Voltage Scaling	15-25%	Dynamic workload environments	Balances power and performance based on demand	Requires precise voltage control circuitry
Power Gating	20-30%	Long idle periods	Fully disconnects inactive modules	Transition delays may impact performance
Clock Gating	15-20%	Pipeline stages in RISC cores	Reduces unnecessary toggling in idle stages	Only affects dynamic power; limited impact on leakage
Multi-Threshold CMOS (MTCMOS)	25-40%	Sleep modes	High-threshold transistors reduce leakage in sleep	May add complexity to circuit design
Dynamic Voltage and Frequency Scaling (DVFS)	20-30%	Variable workloads	Adjusts voltage and frequency in real-time for energy savings	Complex control mechanisms; can affect processing speed
Modified Harvard Architecture	12-15%	Separates instruction and data memory, allowing parallel data access	Reduces memory conflicts, enhances access efficiency	Adds complexity in memory organization

2.3 Design Methodologies

The design of low power 8-bit RISC microcontrollers often employs hardware description languages (HDLs) such as VHDL and Verilog. These languages facilitate the modeling, simulation, and synthesis of microcontroller designs, allowing designers to evaluate power consumption, performance, and area (PPA) trade-offs early in the design process. The adoption of high-level synthesis tools has also streamlined the design workflow, enabling designers to focus on architectural innovations while automating the lower-level implementation details [6], [7], [11].

Architectural innovations play a significant role in enhancing the performance and power efficiency of 8-bit RISC microcontrollers. Techniques such as pipelining and superscalar execution have been adapted for simpler architectures to improve instruction throughput without significantly increasing power consumption [10], [13]. Furthermore, the integration of multi-core designs, even in low-power microcontrollers, has been explored to achieve higher performance levels while maintaining energy efficiency. These design methodologies not only improve the overall functionality of the microcontrollers but also ensure that they remain competitive in a rapidly evolving technological landscape [9], [12].

Design for Testability (DFT) is another crucial aspect of the design process, particularly for low-power microcontrollers. DFT techniques enhance the reliability of microprocessor designs by facilitating easier verification and debugging processes. By incorporating built-in self-test (BIST) mechanisms and scan chain architectures, designers can ensure that the microprocessor operates correctly under various conditions, which is vital for applications in safety-critical systems [19]. The emphasis on DFT in the design phase helps to mitigate risks associated with design errors and improves the overall robustness of the final product [14].

2.4 Verification Techniques

Formal verification methods are increasingly being utilized to ensure the correctness of microprocessor designs. Techniques such as model checking and theorem proving provide a mathematical foundation for verifying that a design meets its specifications. These methods are particularly valuable in low-power designs, where the complexity of power management features can introduce subtle bugs that may not be caught through traditional testing methods.

Recent studies have highlighted the effectiveness of formal verification in identifying design flaws early in the development process, thereby reducing the time and cost associated with post-silicon debugging [17], [21].

Simulation-based verification remains a cornerstone of the verification process for microprocessors. This approach involves creating detailed models of the microprocessor and running extensive test cases to validate functionality and performance. Tools such as ModelSim and Cadence are commonly used for this purpose, allowing designers to simulate various operating conditions and workloads [15], [20]. The integration of power-aware simulation techniques has also gained traction, enabling designers to assess the power consumption of their designs under different scenarios. By combining formal and simulation-based verification techniques, designers can achieve a higher level of confidence in the reliability and efficiency of their low-power 8-bit RISC microprocessors [18], [22].

2.5 Highlights of the Survey

The survey conducted on synthesizable low-power 8-bit RISC microcontrollers reveals several critical insights and trends in the field of microcontroller design and verification. The following highlights encapsulate the key findings from the review of 30 significant research studies:

1. **RISC Architecture Efficiency:** The RISC (Reduced Instruction Set Computing) architecture is identified as particularly suitable for low-power applications due to its simplified instruction set and reduced control logic. This efficiency allows for faster execution of instructions and a more effective use of resources, which is crucial for battery-operated devices.
2. **Power Management Techniques:** The survey emphasizes various power management techniques that have been successfully implemented in low-power microprocessors. Key techniques include:
 - **Clock Gating:** This method selectively disables clock signals in inactive components, achieving power savings of up to 20-25%. Advanced versions, such as deterministic clock gating, can further enhance power efficiency.

- **Power Gating:** By completely shutting off power to inactive units, power gating can reduce leakage current significantly, with reported savings of 20-30% during idle periods.
 - **Dynamic Voltage and Frequency Scaling (DVFS):** This technique adjusts the operating voltage and frequency based on workload demands, leading to power reductions of 20-30% while maintaining performance levels.
3. **Design Methodologies:** The use of Hardware Description Languages (HDLs) such as VHDL and Verilog is prevalent in the design of low-power 8-bit RISC microprocessors. These languages facilitate modeling, simulation, and synthesis, allowing designers to evaluate power-performance-area (PPA) trade-offs early in the design process.
 4. **Verification Techniques:** The survey highlights the importance of robust verification methodologies in ensuring the correctness of microprocessor designs. Formal verification methods, such as model checking and theorem proving, are increasingly utilized to catch design flaws early in the development cycle. Simulation-based verification remains a cornerstone, with tools like ModelSim and Cadence being commonly employed.
 5. **Performance Metrics and Trade-offs:** The analysis underscores the necessity of evaluating power-performance trade-offs in microprocessor design. Improvements in these areas can lead to significant power savings while sustaining performance levels, which is essential for resource-constrained applications.
 6. **Emerging Trends:** The survey identifies emerging trends in microprocessor design, including the integration of AI and machine learning for adaptive power management and real-time optimization. Advanced fabrication technologies, such as FinFET and 3D stacking, are also noted for their potential to enhance performance and energy efficiency.
 7. **Educational Value:** The 8-bit RISC architecture serves as an effective educational tool for understanding computer architecture, bridging theoretical concepts with practical implementation. This simplicity makes it an ideal choice for academic purposes, reinforcing foundational concepts like pipelining and memory access.

CHAPTER 3

DESIGN SPECIFICATION AND METHODOLOGY

Chapter 3

DESIGN SPECIFICATION AND METHODOLOGY

This project leverages the Reduced Instruction Set Computer (RISC) architecture, chosen for its simplicity and efficiency in processing, which reduces execution time. The RISC architecture enables the processor to handle essential operations with minimal instruction cycles, enhancing energy efficiency by avoiding unnecessary complexity. By using a 4-bit opcode, the instruction set remains compact, allowing the microcontroller to execute tasks quickly with fewer transistors, which translates to lower power usage.

Table 3.1 provides a detail information about Design Specifications of RISC Microcontroller.

Table 3.1: - Design Specifications of RISC Microcontroller

Specification Category	Specification
Architecture	8-bit RISC
Data Width	8 bits
Address Width	4 bits (16 locations of memory)
Registers	8-bit Accumulator, 8-bit B Register, 4-bit PC, 8-bit IR
ALU Width	8 bits
Pipeline Stages	1 stage (no pipelining, sequential execution, Ring Counter (6 Cycles)
RAM Capacity	16 bytes (16 locations × 8 bits in SRAM)
Number of I/O Ports	One 8-bit parallel I/O port (via DATA_OUT)
Data Bus	8-bit bidirectional
Address Bus	4-bit bidirectional
Control Bus	Read, Write (basic control signals like CE, LM, CP, EP, LI, EI, LA, EA, LB, LO, NOP, ALU_OP)

3.1 INSTRUCTIONS SET

Table 3.2: - Instructions Set

Opcode	Instructions	Source	Destination	Bytes	Description
00H	LDA	addr	A	1	Loads data from the address into the accumulator (A).
01H	ADD	addr	A	1	Adds data from the address to A and stores the result in A.
02H	SUB	addr	A	1	Subtracts data from the address from A and stores the result in A.
03H	AND	addr	A	1	Performs AND logic between addr and A, result stored in A.
04H	OR	addr	A	1	Performs OR logic between addr and A, result stored in A.
05H	XOR	addr	A	1	Performs XOR logic between addr and A, result stored in A.
06H	NOT		A	1	Inverts the bits of A and stores the result in A.
07H	SHL		A	1	Shifts A left by 1 bit and stores the result in A.
08H	SHR		A	1	Shifts A right by 1 bit and stores the result in A.
09H	MUL	addr	A	1	Multiplies A with data from addr and stores the result in A.
0AH	DIV	addr	A	1	Divides A by data from addr and stores the result in A (0 if divide by zero).
0BH	INC		A	1	Increments A by 1 and stores the result in A.
0CH	DEC		A	1	Decrements A by 1 and stores the result in A.

Opcode	Instructions	Source	Destination	Bytes	Description
0DH	CMP	addr		1	Compares A with data from addr (result not stored, affects flags implicitly).
0EH	OUT			1	Outputs the value of A to the output register.
0FH	HLT			1	Halts the CPU execution.

Table 3.2 provides an in depth information about the instructions which are being used in the microcontroller. The microcontroller design includes critical internal components such as a Control Unit, Arithmetic Logic Unit (ALU), Register File, and etc. These units work in tandem through a ring counter operation balancing throughput and power efficiency.

The Control Unit is responsible for managing the overall operation of the microcontroller by coordinating and controlling the activities of all other components. It decodes the opcode of each instruction fetched from memory, generating control signals that direct the execution of the instruction. In a RISC architecture, the Control Unit is typically designed to be simple, enabling quick instruction decoding and control signal generation.

The Arithmetic Logic Unit is the core component responsible for performing all arithmetic and logical operations. In an 8-bit RISC processor, the ALU handles operations such as addition, subtraction, AND, OR, XOR, and NOT.

The Register File is a small, fast storage area that holds the general-purpose registers. In an 8-bit microcontroller, each register is 8 bits wide and is used to store data temporarily during instruction execution.

The ring counter implemented (with 6 states) is used to generate control signals for the various stages of instruction execution. Each state corresponds to a specific step in the instruction cycle. The ring counter cycles through its states, and each state can trigger specific control signals that dictate the behaviour of the CPU components (like the ALU, memory, etc.).

In this model, instructions are executed one after the other in a sequential manner. Each instruction goes through a series of defined states (controlled by the ring counter) before it is fully executed. The execution of an instruction is broken down into multiple steps, and the CPU waits for the completion of each step before moving to the next. The control unit (CU) generates control signals based on the current state of the ring counter.

These signals control various components of the CPU, such as loading data into registers, performing ALU operations, and accessing memory. The control signals are activated in a specific sequence, ensuring that each part of the CPU operates in the correct order.

Various power optimization techniques are incorporated into the design. Clock gating minimizes dynamic power by stopping the clock in inactive sections, while power gating completely cuts off power to non-essential areas when they are not in use. Additionally, dynamic voltage scaling is employed to adjust the operating voltage according to workload, further reducing power during periods of low activity. These strategies are pivotal in extending battery life, which is essential for applications in IoT and portable electronics.

Clock Gating: This technique disables the clock signal to inactive components, reducing dynamic power consumption by preventing unnecessary switching activity.

Power Gating: Completely turns off power to non-active blocks, eliminating both static and dynamic power leakage when the components are not in use.

Dynamic Voltage Scaling (DVS): Adjusts the supply voltage according to workload requirements, lowering voltage during idle periods to decrease power usage.

3.2 Software Requirements

Xilinx ISE (Integrated Software Environment) is a powerful development suite used primarily for designing, synthesizing, simulating, and implementing digital circuits on Xilinx FPGAs. ISE allows users to work with HDLs such as Verilog and VHDL to create and test complex digital designs. The tool includes components like the ISE Simulator, which helps verify functional correctness, and Xilinx Synthesis Technology (XST), which translates HDL code into a gate-level representation optimized for Xilinx devices. Xilinx ISE also provides advanced optimization options to enhance speed and area efficiency, making it suitable for prototyping digital circuits and fine-tuning FPGA-based systems.

Cadence Genus is a comprehensive tool for synthesis and design implementation in digital circuit design, particularly for ASICs (Application-Specific Integrated Circuits). Genus employs advanced synthesis algorithms and optimization techniques to translate high-level HDL code into an optimized gate-level netlist. With features like multi-objective optimization, physical-aware synthesis, and power-efficient design techniques, Genus enables designers to achieve high-performance, low-power, and area-efficient designs.

The tool integrates well with Cadence's suite of EDA tools, such as Innovus for physical design and Joules for power estimation, to provide a seamless flow from RTL (Register TransferLevel) code to tape-out. Genus supports a broad range of applications, from standard ASICs to custom low-power designs, making it valuable in various high-complexity design environments.

3.3 Design Methodology

The outlined method describes a systematic approach for the CPU to process instructions through a series of well-defined stages. Each step is critical to ensure that the instruction is executed correctly and efficiently, transforming high-level programs into machine-level operations effectively. The figure outlines the steps a CPU takes to execute an instruction, starting from fetching it from memory to writing back the result.

The Figure 3.1 presents a flowchart detailing the operation of a basic instruction cycle in a computer system. At the top of the diagram, we see the Program Counter, which is responsible for holding the address of the next instruction to be executed. This counter plays a crucial role in ensuring that the processor fetches instructions in the correct sequence.

The flow begins with the Instruction Fetch, where the next instruction is retrieved based on the address provided by the Program Counter. Control signals are also generated to manage this process. Once the instruction is fetched, the next step is Instruction Decode, which involves interpreting the instruction to determine the required operations and the operands involved.

Following the decoding phase, the system engages the Arithmetic Logic Unit (ALU). This component performs computations and logical operations based on the decoded instruction and provided operands. The result from the ALU is then processed in the Execution stage, where the intended action of the instruction is carried out.

The next significant phase is Memory Access, which allows the system to either read from or write to the memory. Depending on the instruction type, this could involve loading data into the register or storing data back to memory. After memory operations are completed, the cycle progresses to the Write Back stage, where the results are written to the Register File.

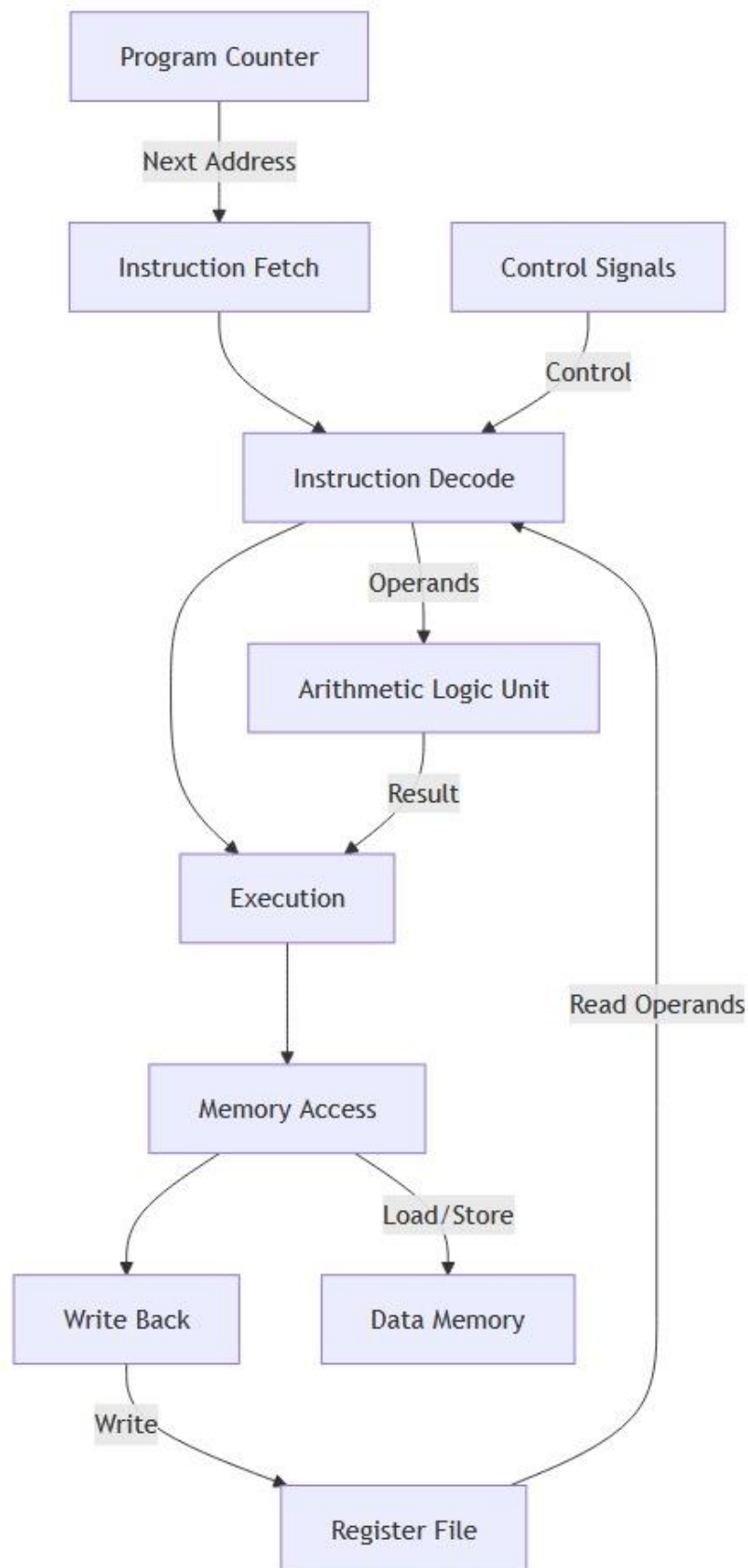


Figure 3.1: - Flow Diagram of RISC Microcontroller

Overall, this flowchart encapsulates the fundamental sequence of operations that a computer's processor undertakes to perform instructions, highlighting the interconnected stages from fetching an instruction to writing back results. Each step is essential for the effective execution of software instructions, underpinning the complexity of modern computing.

CHAPTER 4

LOW LEVEL PROCESSOR DESIGN MODULE IMPLEMENTATION

Chapter 4

LOW LEVEL PROCESSOR DESIGN MODULE IMPLEMENTATION

This chapter details the implementation of the 8-bit Reduced Instruction Set Computer (RISC) microcontroller using the Xilinx design tools. It covers the design process, the components developed.

4.1 Microcontroller Components

1. Verification Arithmetic Logic Unit (ALU):

The ALU is responsible for performing arithmetic and logical operations. It supports operations such as addition, subtraction, AND, OR, NOT, etc. The ALU takes two input operands and a control signal that determines the operation to be performed. Figure 4.1 represents the Arithmetic Logic Unit schematic block.

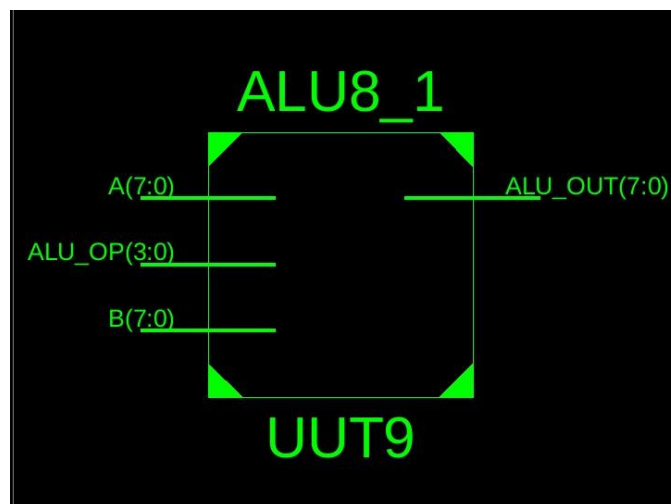


Figure 4.1: - Arithmetic Logic Unit Schematic

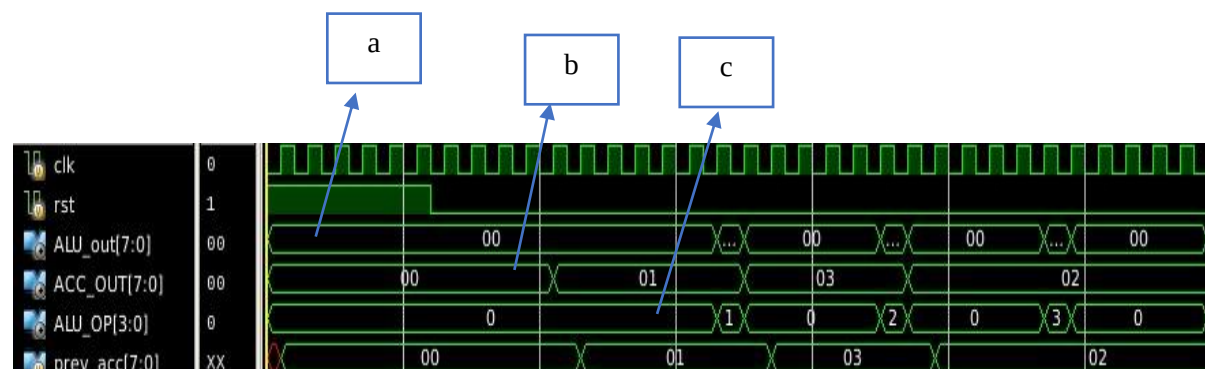


Figure 4.2: - Arithmetic Logic Unit Waveform

Figure 4.2 represents the Arithmetic Logic unit waveform and explanation for markers are given below: -

- a.) **ALU_OUT:** -It provides the results of the operations performed based on the opcode.
- b.) **ACC_OUT:** - It is the storing register of all the operations results and u as the operand for next operation.
- c.) **ALU_OP:** - It is a 4-bit opcode used to identify the operation based on the instruction fetched.

2. Verification Program Counter (PC):

The Program Counter holds the address of the next instruction to be executed. It is incremented after each instruction fetch, ensuring the sequential execution of instructions.

Figure 4.3 represents the schematic block of the Program Counter module.

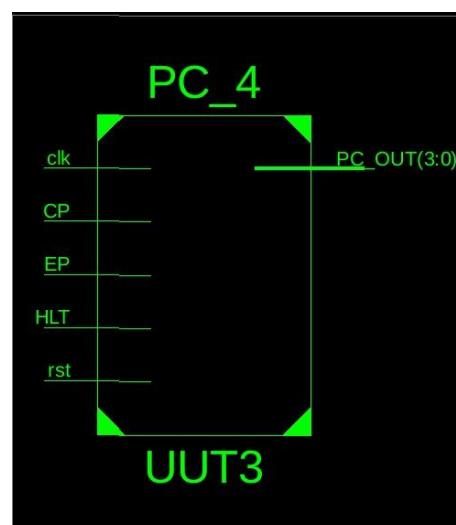


Figure 4.3: -Program Counter Schematic

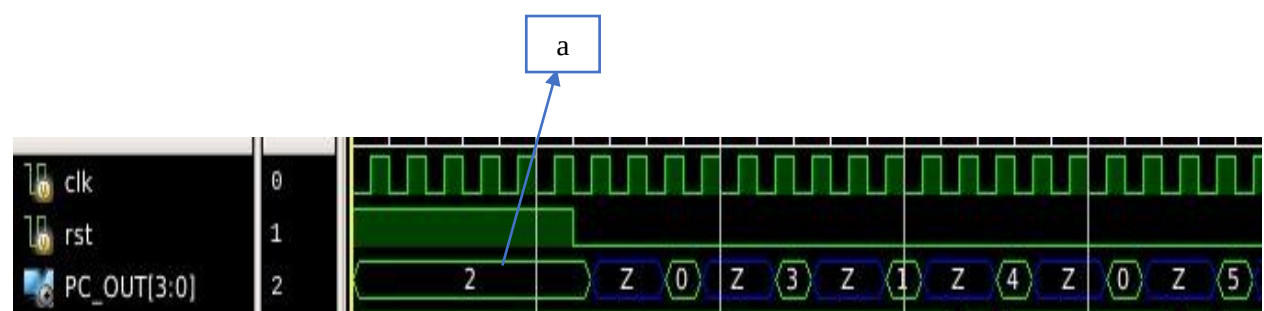


Figure 4.4: - Program Counter Waveform

Figure 4.4 represents the waveform of the program counter with its output and the explanation of the marker is given below.

- a.) **PC_OUT:** - It helps to fetch the next instructions by incrementing the address by 1 whenever it is enabled and the instruction is fetched after the ring counter has processed through 6 stages.

3. Verification Control Unit (CU):

The Control Unit orchestrates the operation of the microcontroller by generating control signals based on the current instruction being executed. It decodes the instruction fetched from memory and directs the ALU, registers, and other components to perform the necessary operations. The CU ensures that the microcontroller operates in a coordinated manner. Figure 4.5 represents the schematic block of the Control unit module.

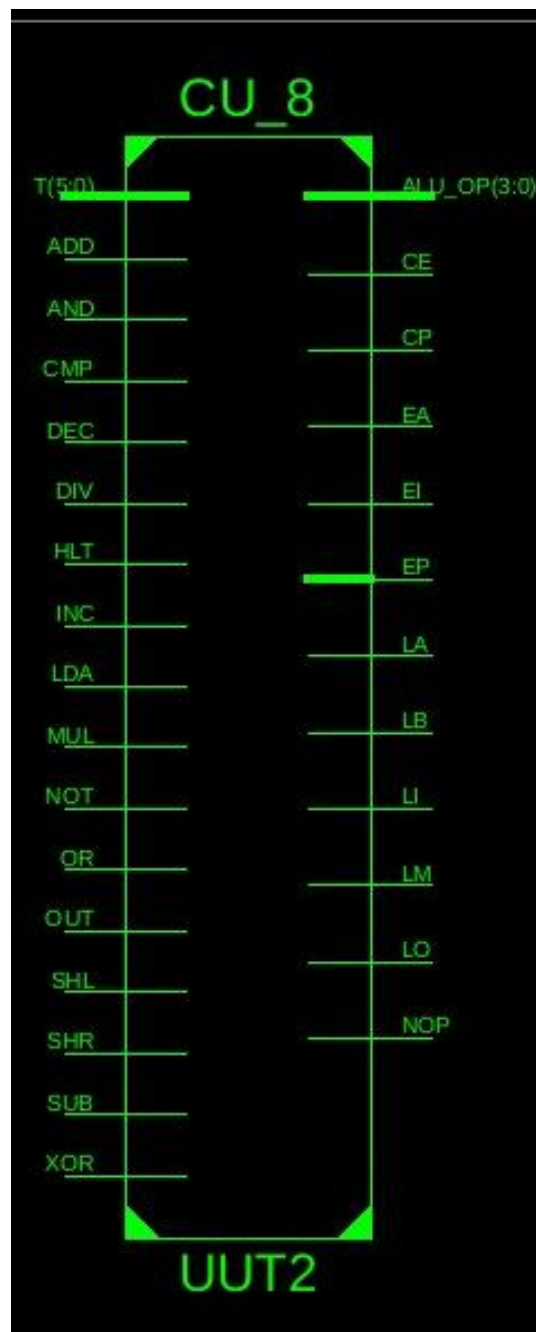


Figure 4.5: -Control Unit Schematic

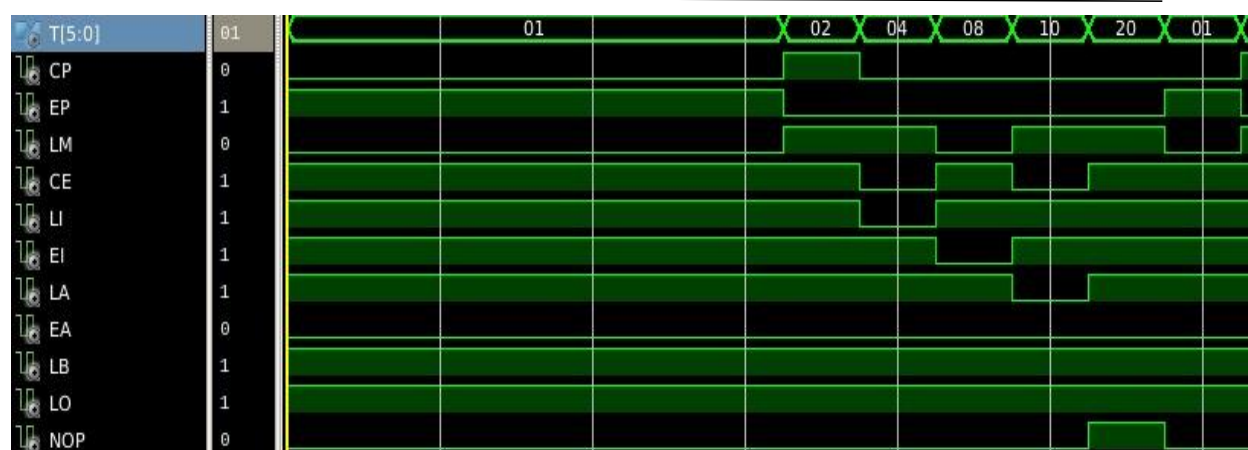


Figure 4.6: -Control Unit Waveform

Figure 4.6 represents the waveform of control unit with its output and the explanation of the signals/ variables has been given below

- a) **T[5:0]:** The timing signal shifts from 000001 (T1) to 100000 (T6) and repeats. Mark each bit's active period.
- b) **CP (Counter Pulse):** Active on T2 when not halted (!HLT). Mark as 1 at T2 cycles (e.g., 02, 04, 08, 10).
- c) **EP (Enable PC):** Active on T1. Mark as 1 at T1 cycles (e.g., 01).
- d) **LM (Load Memory Address):** Active on !T1 and !(T4 & (LDA | ALU_MEM)). Mark as 1 when T1 is low and T4 is low or no memory operation.
- e) **CE (Chip Enable):** Active on !T3 and !(T5 & (LDA | ALU_MEM)). Mark as 1 when T3 is low and T5 is low or no memory operation.
- f) **LI (Load Instruction):** Active on !T3. Mark as 1 when T3 is low (e.g., 04, 08).
- g) **EI (Enable Instruction):** Active on !(T4 & (LDA | ALU_MEM)). Mark as 1 when T4 is low or no memory operation.
- h) **LA (Load Accumulator):** Active on !(T5 & LDA) or (T6 & (ALU_MEM | ALU_NO_MEM)). Mark as 1 at T5 for LDA or T6 for ALU operations.
- i) **EA (Enable Accumulator):** Active on T4 & OUT. Mark as 1 at T4 when OUT is active.
- j) **LB (Load B Register):** Active on !(T5 & ALU_MEM). Mark as 1 when T5 is low or no memory ALU operation.
- k) **LO (Load Output):** Active on !(T4 & OUT). Mark as 1 when T4 is low or no output operation.
- l) **NOP (No Operation):** Active on (T6 & (LDA | ALU_MEM | ALU_NO_MEM | HLT)) or (T5 & OUT). Mark as 1 at T6 for instructions or T5 for OUT.

5. Verification Memory Address Register (MAR):

The Memory Address Register (MAR) is a crucial component of the microcontroller architecture that plays a vital role in the memory access process. It holds the address of the memory location that the microcontroller intends to read from or write to. The MAR acts as a bridge between the microcontroller and the memory unit, ensuring that the correct data is accessed during instruction execution. Figure 4.7 represents the schematic block of the Memory Address Register.

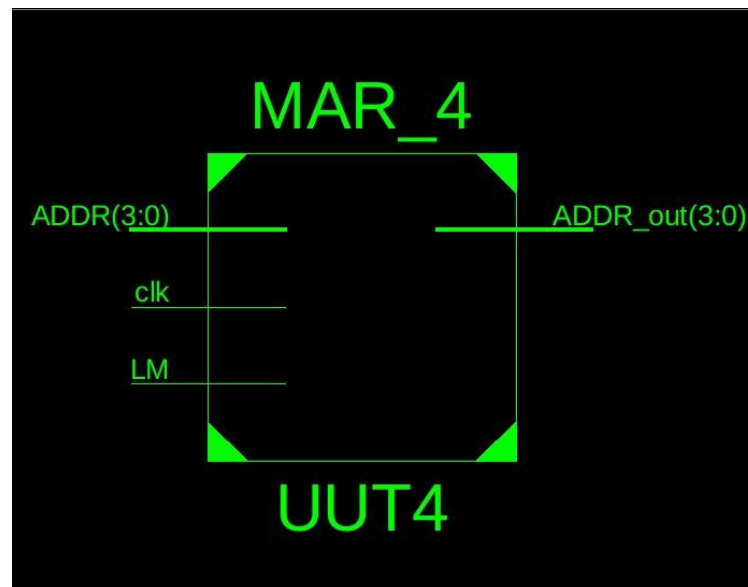


Figure 4.7: -Memory Address Register Schematic

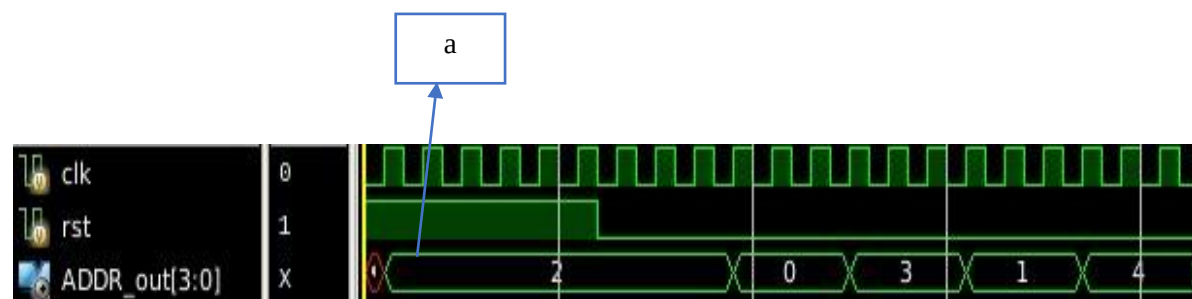


Figure 4.8: -Memory Address Register Waveform

Figure 4.8 represents the waveform of Memory Address Register with its output and the explanation of the marker been given below

- a.) **ADDR_out:** - It is the output of the memory address register, an address is fetched whenever load memory is enabled and ADDR is the input of the memory address register.

6. Verification of SRAM

Static Random-Access Memory (SRAM) serves as a critical component for data storage and retrieval. SRAM is a type of volatile memory that retains data as long as power is supplied, making it ideal for applications requiring fast access times and low latency.

The SRAM in the Veonumen architecture is utilized for various purposes, including instruction storage, data caching, and temporary data storage during processing.

Its ability to provide quick read and write access allows the microcontroller to efficiently execute instructions and manage data operations, significantly enhancing overall system performance. Figure 4.9 represents the schematic block of the Static Random-Access Memory module.

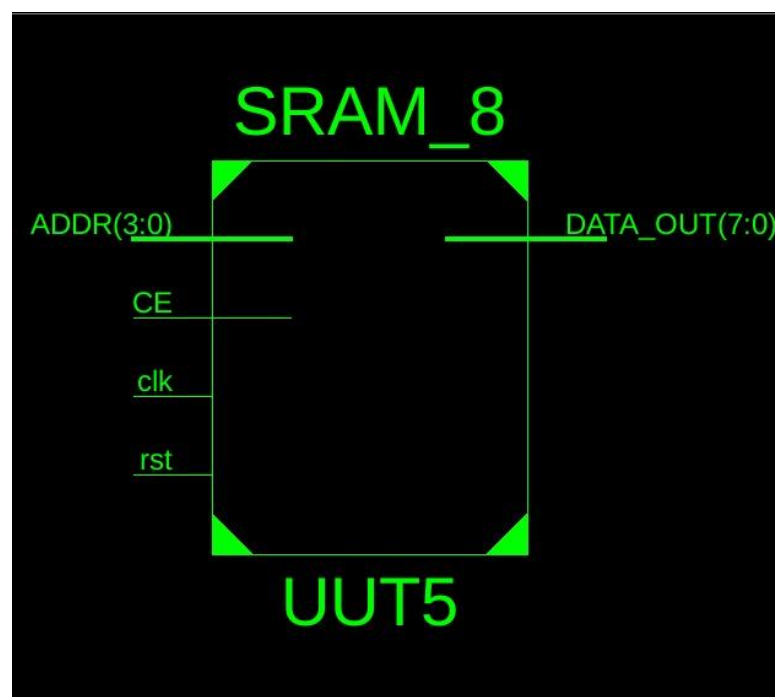


Figure 4.9: - Static Random-Access Memory Schematic

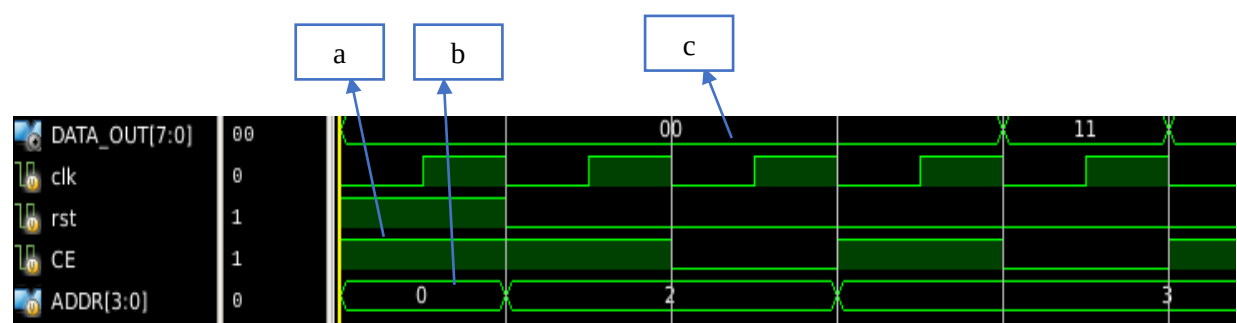


Figure 4.10: - Static Random-Access Memory Waveform

Figure 4.10 represents the waveform of Static Random-Access Memory with its output and the explanation of the markers has been given below.

- a) **Chip Enable (CE):** - Controls whether the SRAM outputs data. Active low ($\sim$ CE), so mark CE = 0 when data is expected to be output.
- b) **Address (ADDR [3:0]):** - The 4-bit address input selects the memory location. Mark the address values as they change (e.g., 0, 2, 12).
- c) **Data Output (DATA_OUT [7:0]):** - The 8-bit data output from the SRAM based on the address and CE. Mark the data values corresponding to the SRAM content (e.g., 00, 11).

7. Verification of Input Register (IR)

The Input Register (IR) is a fundamental component of the Veonumen architecture that plays a crucial role in the instruction execution process. The IR is responsible for temporarily holding the instruction that has been fetched from memory before it is decoded and executed by the microcontroller. This register acts as a buffer between the memory unit and the control unit, ensuring that instructions are processed efficiently

When the microcontroller retrieves an instruction from memory, it is first loaded into the IR. This allows the control unit to access the instruction quickly without needing to repeatedly access the slower memory. The IR holds the instruction until it is fully decoded, enabling the microcontroller to determine the necessary operations and the data involved in executing the instruction. Figure 4.11 represents the schematic block of the Instruction Register module.

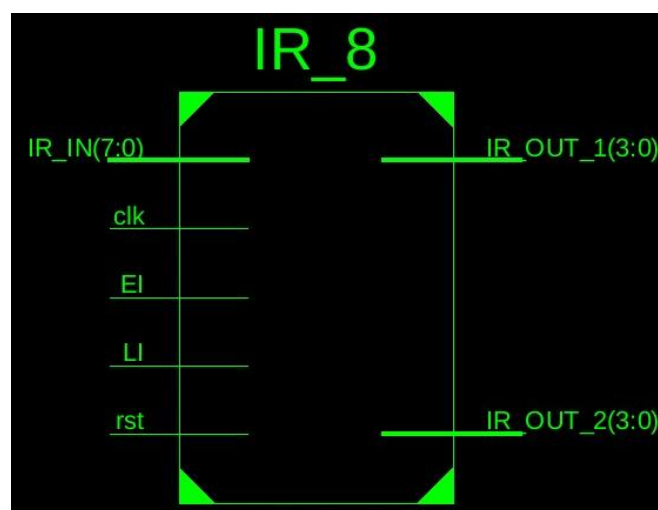


Figure 4.11: - Instruction Register Schematic

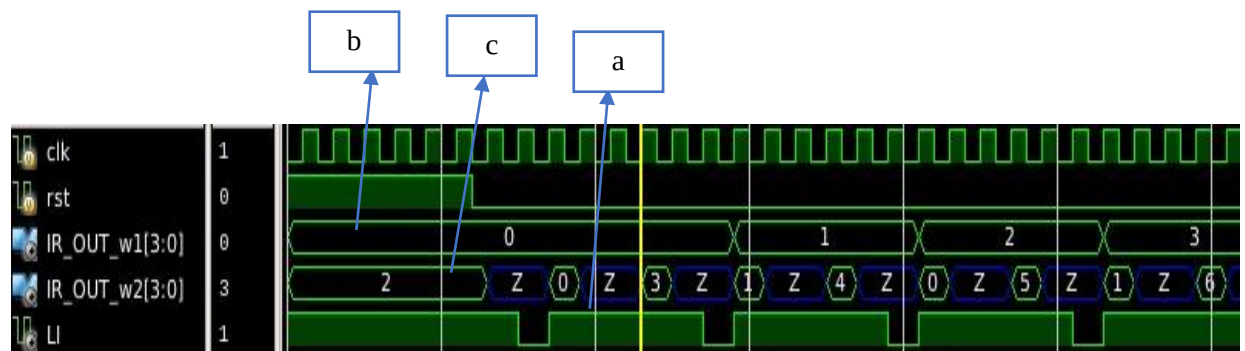


Figure 4.12: - Instruction Register Waveform

Figure 4.12 represents the waveform of Instruction Register with its output and the explanation of the markers has been given below.

- a) **Load Instruction (LI):** Starts at 1, transitions to 0 at time 2, and back to 1 at time 4. This controls loading into the Instruction Register (IR₈).
- b) **IR_OUT_w1[3:0]:** Starts at 0, changes to 3 at time 2, and remains 3. This is the upper 4 bits of the instruction register.
- c) **IR_OUT_w2[3:0]:** Starts at 0, changes to 2 at time 2, then follows a sequence (0, 3, 1, 4, 0, 5, 1, 6) after time 4. This is the lower 4 bits of the instruction register when EI is not active.

8. Verification of Ring Counter (RC)

The Ring Counter is a critical component of the microcontroller architecture that plays a vital role in the instruction execution process. It is used to generate control signals for the various stages of instruction execution. Each state corresponds to a specific step in the instruction cycle. The Ring Counter cycles through its states, and each state can trigger specific control signals that dictate the behavior of the CPU components (like the ALU, memory, etc.).

The waveform of the Ring Counter shows the 6-stage transition from 1 to 6, indicating the correct operation of the Ring Counter. The Ring Counter is essential for ensuring that the microcontroller operates in a coordinated manner, and its correct operation is crucial for the overall functionality of the microcontroller. Figure 4.13 represents the schematic block of the Ring Counter module.

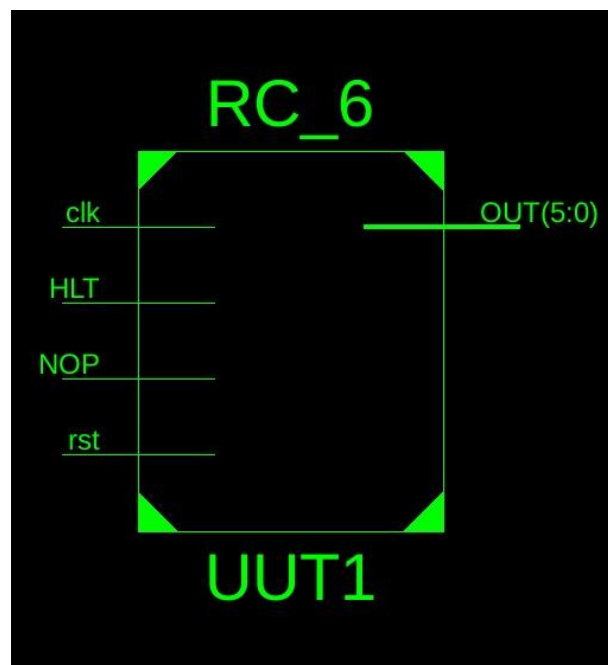


Figure 4.13: - Ring Counter Schematic

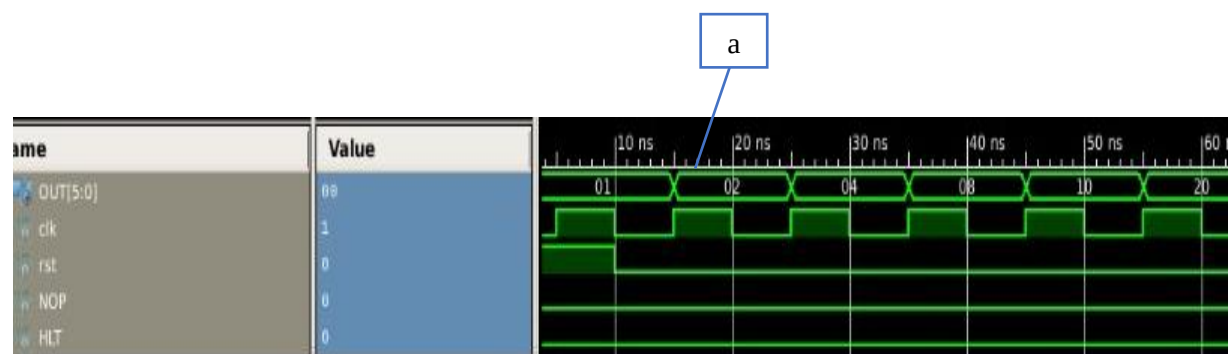


Figure 4.14: - Ring Counter Waveform

Figure 4.14 represents the waveform of Instruction Register with its output and the explanation of the markers has been given below.

a.) **OUT:** It shows the 6-stage Ring Counter transition from 1 to 6.

9. Verification of Accumulator

The Accumulator is a fundamental component of the Veonumen architecture, serving as a specialized register that temporarily holds intermediate results of arithmetic and logic operations performed by the microcontroller. It plays a crucial role in the execution of instructions, particularly those involving calculations, data manipulation, and storage of results.

When the microcontroller executes an instruction that requires computation, the operands are typically loaded into the accumulator, and the result of the operation is also stored in the accumulator. Figure 4.15 represents the schematic block of the Accumulator module.

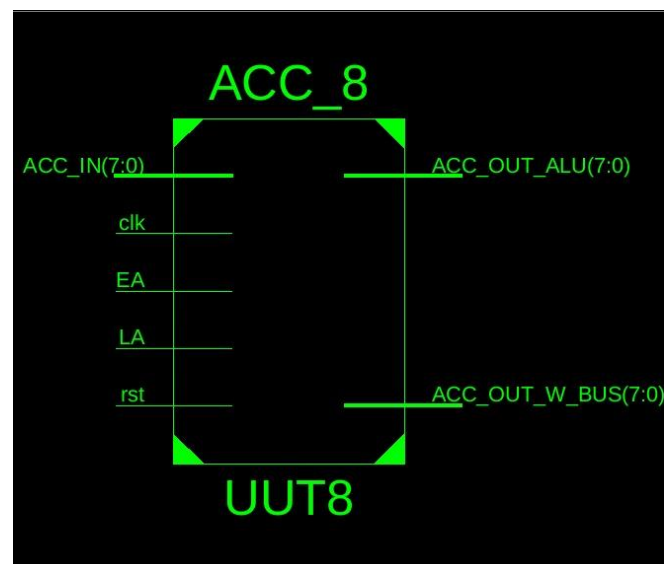


Figure 4.15: -Accumulator Schematic

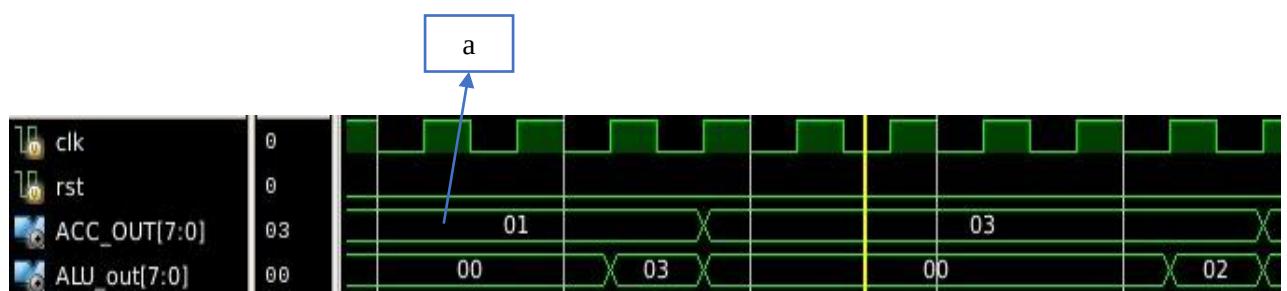


Figure 4.16: -Accumulator Waveform

Figure 4.16 represents the waveform of Instruction Register with its output and the explanation of the marker has been given below

a.) **ACC_OUT**: - It accumulates or stores the results of the ALU operations.

10. Verification of Output Register (OR)

The Output Register (OR) is a critical component of the Veonumen architecture, responsible for temporarily holding data that is to be sent from the microcontroller to external devices or memory. It acts as a buffer between the microcontroller and the output peripherals, ensuring that data is transmitted efficiently and reliably.

When the microcontroller completes a computation or data manipulation task, the resulting data is loaded into the output register before being sent to its destination, such as a display, printer, or other output devices. Figure 4.17 represents the schematic block of the Output Register module.

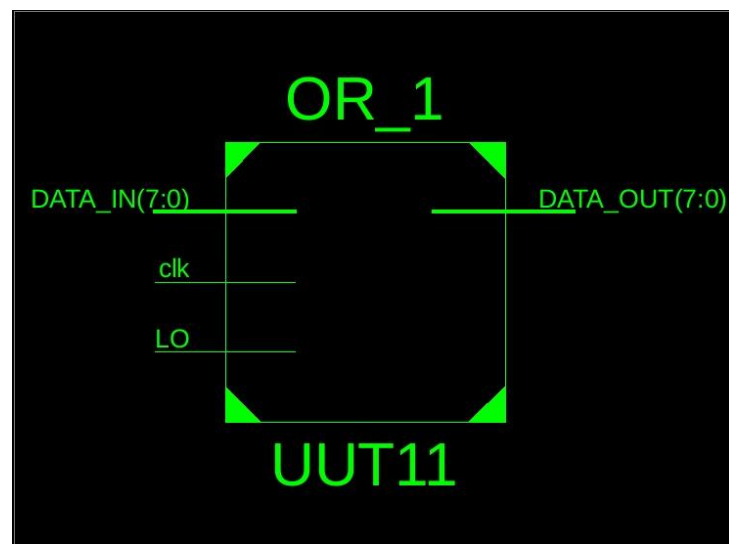


Figure 4.17: - Output Register Schematic

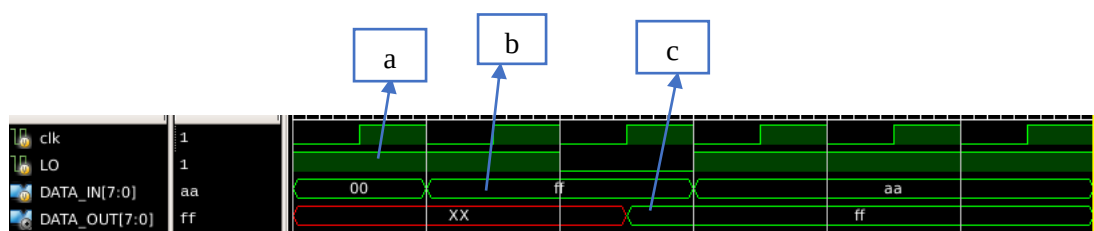


Figure 4.18: - Output Register Waveform

Figure 4.18 represents the waveform of Output Register with its output and the explanation of the markers has been given below.

- a.) **Load Output (LO):** Starts at 0, transitions to 1 briefly, then back to 0. This signal controls the loading of data into the output register (OR_1).
- b.) **DATA_IN[7:0]:** Starts with a value of aa (hex, likely 10101010 in binary), changes to XX (unknown/high-impedance) during the LO pulse, and returns to aa afterward.
- c.) **DATA_OUT[7:0]:** Starts with 00 (likely 00000000 in binary), changes to ff (11111111 in binary) after the LO pulse, and remains ff thereafter.

11. Verification of Instruction Decoder (ID)

The Instruction Decoder (ID) is a vital component of the Veonumen architecture, responsible for interpreting and translating the binary instructions fetched from memory into control signals that direct the operation of the microcontroller. The ID plays a crucial role in the instruction execution cycle, ensuring that the microcontroller understands and executes the intended operations correctly.

When an instruction is loaded into the Input Register (IR), the instruction decoder analyzes the binary representation of the instruction to determine its type and the specific operations to be performed. This process involves decoding the opcode (operation code) and identifying the operands involved in the instruction. The output of the instruction decoder consists of control signals that guide various components of the microcontroller, such as the Arithmetic Logic Unit (ALU), registers, and memory. Figure 4.19 represents the schematic block of the Instruction Decoder module.

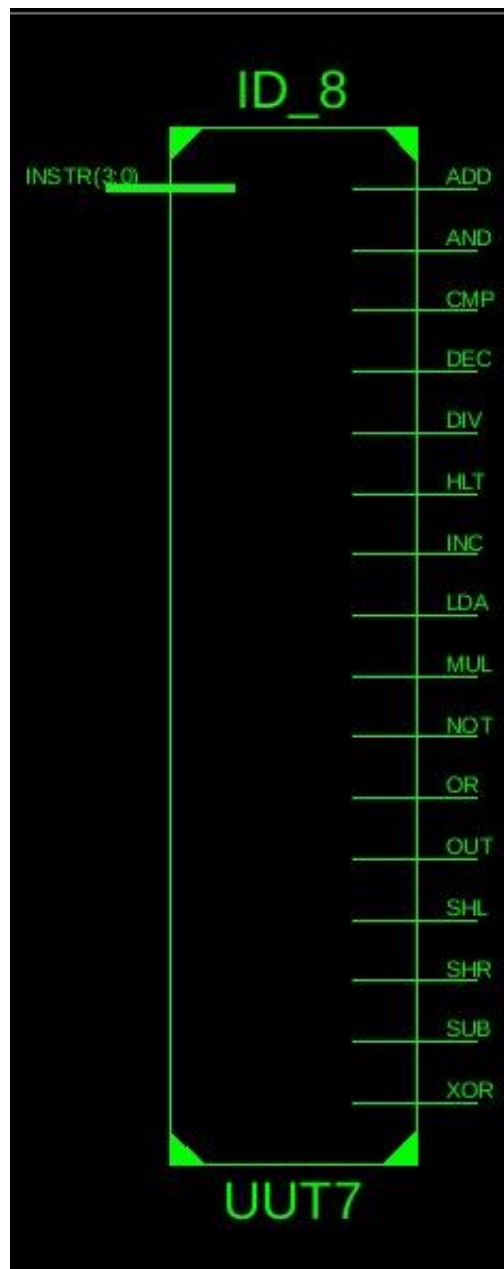


Figure 4.19: - Instruction Decoder Schematic



Figure 4.20: - Instruction Decoder Waveform

Figure 4.20 represents the waveform of Instruction Decoder with its output and the explanation of the marker has been given below

- a) **INSTR[3:0]:** This is a 4-bit input representing the instruction code. The waveform shows:
- Starts with 1111 (binary, likely an invalid or HLT instruction).
 - Changes to 0000 (LDA instruction).
 - Then to 0001 (ADD instruction).
 - Finally to 0010 (SUB instruction).

4.2 Low power Techniques

In the design of microcontrollers, especially for low-power applications such as embedded systems and portable electronics, implementing effective low-power techniques is crucial. These techniques aim to minimize power consumption while maintaining performance, thereby extending battery life and reducing thermal output. Three prominent low-power techniques are clock gating, power gating, and dynamic voltage scaling (DVS).

1. Clock Gating

Clock gating is a widely used technique that reduces dynamic power consumption in digital circuits by selectively disabling the clock signal to inactive components of the microcontroller. Since the power consumed by a circuit is proportional to the switching activity, minimizing unnecessary clock toggling can lead to significant power savings.

In a microcontroller, various components (such as registers, ALUs, and memory units) may not be active at all times. Clock gating involves the use of additional logic to control the clock signal. When a component is not in use, the clock signal to that component is turned off, preventing it from switching states and thus reducing dynamic power consumption.

Benefits:

- **Power Savings:** Clock gating can achieve power reductions of 15-25% in many designs, depending on the level of activity and the number of components that can be gated.
- **Simplicity:** It can be implemented with relatively simple logic circuits, making it an attractive option for many designs.

Limitations:

- **Latency:** There may be a slight increase in latency when reactivating gated components, which can affect performance in time-sensitive applications.
- **Complexity in Control Logic:** The design of the gating logic can add complexity to the overall circuit design.

Figure 4.21 displays the schematic of the 8-bit microcontroller incorporating clock gating techniques. This schematic provides a visual representation of the microcontroller's architecture, highlighting the integration of clock gating elements designed to optimize power efficiency.

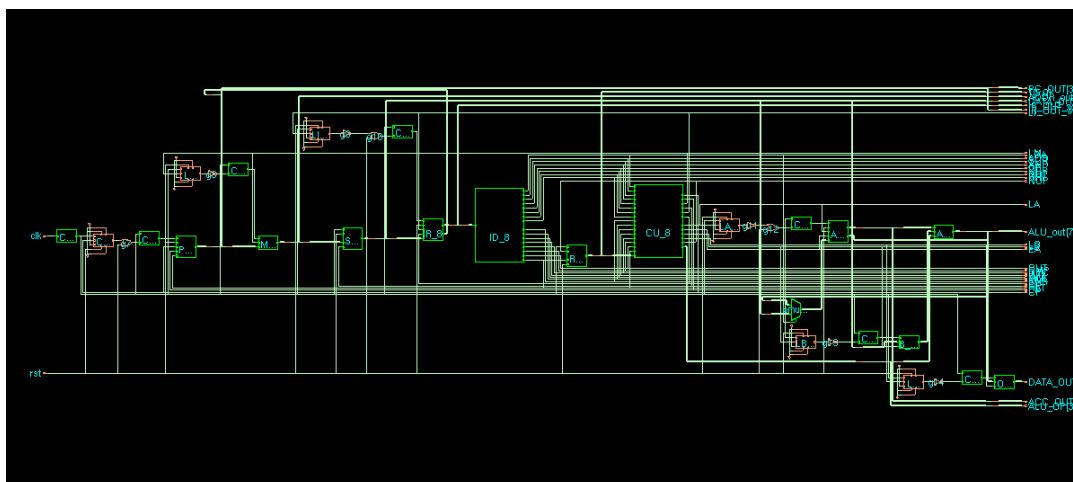


Figure 4.21: - 8-bit Microcontroller with Clock Gating Schematic before synthesis

2. Power Gating

Power gating is a technique that reduces both static and dynamic power consumption by completely shutting off the power supply to inactive blocks of the microcontroller. This is particularly effective in reducing leakage power, which is a significant concern in modern CMOS technologies.

Power gating involves the use of sleep transistors (also known as power switches) that can disconnect the power supply from certain components when they are not in use. When a component is needed again, the power switch is turned on, restoring power to that component.

Benefits:

- **Significant Power Reduction:** Power gating can lead to power savings of 20-30%, especially during idle periods when many components are not in use.
- **Leakage Control:** It effectively reduces leakage current, which is crucial for battery-operated devices.

Limitations:

- **Transition Delays:** The time taken to switch components back on can introduce delays, which may impact performance if not managed properly.
- **Design Complexity:** Implementing power gating requires careful design to ensure that the transition between active and inactive states does not disrupt the operation of the microcontroller.

$$P_{\text{gated}} = (C_{\text{ALU}} \cdot V^2 \cdot f \cdot \alpha_{\text{ALU}}) + (C_{\text{SRAM}} \cdot V^2 \cdot f \cdot \alpha_{\text{SRAM}}) + (C_{\text{ACC}} \cdot V^2 \cdot f \cdot \alpha_{\text{ACC}}) + P_{\text{other}}$$

- **ALU:** Active in T5 for 13 instructions (ADD to CMP).
- **SRAM:** Active in T3 (fetch) for all instructions, and T5 for 8 instructions
- **ACC:** Active in T4



Figure 4.22: - 8-bit Microcontroller with Power Gating Schematic before synthesis

Figure 4.22 displays the schematic of the 8-bit microcontroller incorporating power gating techniques. This schematic visually represents the microcontroller's architecture, highlighting the integration of power gating elements designed to optimize power efficiency.

3. Dynamic Voltage Scaling (DVS)

Dynamic Voltage Scaling (DVS) is a technique that adjusts the supply voltage and frequency of the microcontroller dynamically based on the workload requirements. By lowering the voltage during periods of low activity, DVS can significantly reduce power consumption.

DVS monitors the microcontroller's workload in real time and adjusts the operating voltage and frequency accordingly. When the processor is under heavy load, the voltage and frequency are increased to maintain performance. Conversely, during low-load periods, the voltage and frequency are reduced, leading to lower power consumption.

Benefits:

- **Energy Efficiency:** DVS can achieve power reductions of 20-30% by optimizing voltage and frequency based on workload demands.
- **Performance Management:** It allows for maintaining performance levels during high-demand tasks while conserving energy during lighter workloads.

Limitations:

- **Complex Control Mechanisms:** Implementing DVS requires sophisticated control algorithms to monitor workload and adjust voltage and frequency without introducing latency.
- **Potential for Performance Degradation:** If not managed properly, DVS can lead to performance degradation during critical tasks if the voltage and frequency are not adjusted quickly enough.

$$P_{\text{dynamic}} = C \cdot V^2 \cdot f \cdot \alpha$$

- P_{dynamic} : Dynamic power consumption.
- C : Total capacitance of the circuit (load capacitance).
- V : Supply voltage (VDD_HIGH, VDD_MED, or VDD_LOW).
- f : Clock frequency (inversely related to clock delay in CLOCK_BUFFER_DVS).
- α : Switching activity factor (fraction of gates switching per cycle, depends on workload).

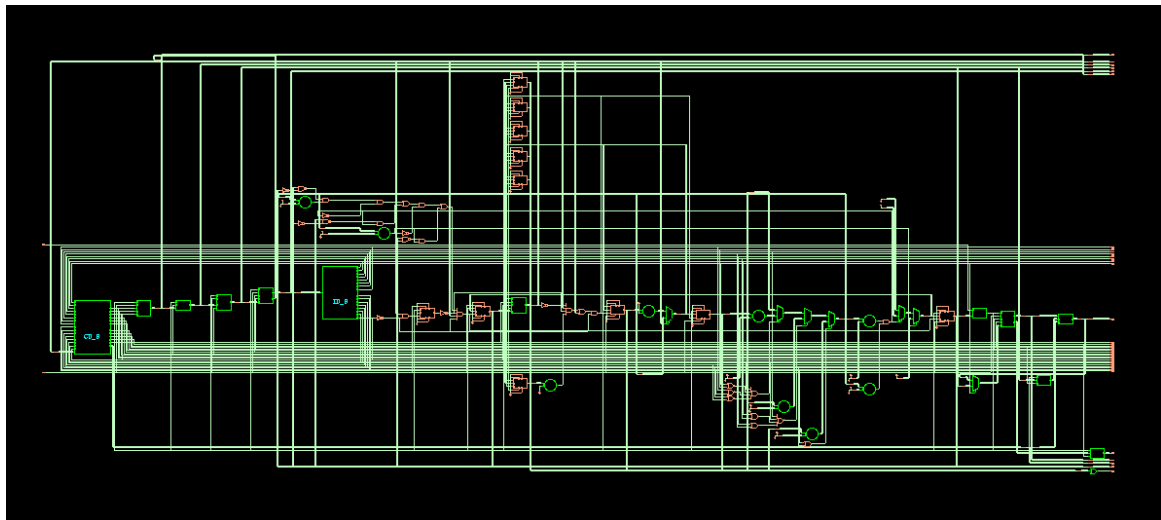


Figure 4.23: - 8-bit Microcontroller with Dynamic Voltage Scaling Schematic before synthesis

Figure 4.23 displays the schematic of the 8-bit microcontroller incorporating dynamic voltage scaling techniques. This schematic visually represents the microcontroller's architecture, highlighting the integration of dynamic voltage scaling elements designed to optimize power efficiency.

CHAPTER 5

RESULTS

Waveform 1 - SimVision

Page 1 of 1

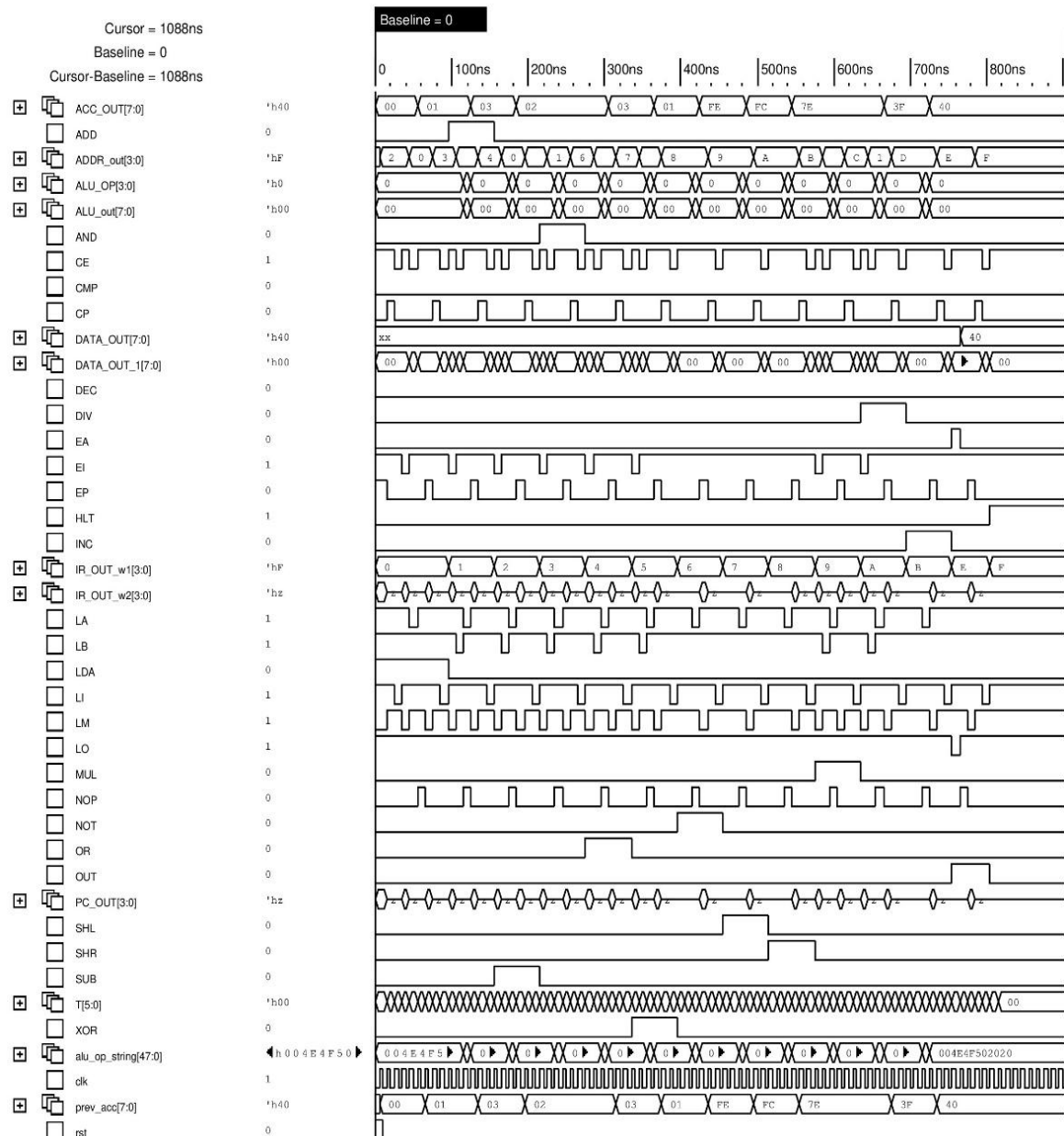


Figure 5.2: - 8-bit Microcontroller Waveform

The different signals of Figure 5.1 are explained as below: -

- a.) ACC_OUT: - Accumulates all the operations outputs.
- b.) DATA_OUT: - After the OUT operation is completed the result will be transferred to the external register
- c.) ALU_OP: -It is the opcode of the ALU operations for the required instruction.
- d.) ADD: -It is One of the operations where it is computed when instruction is fetched

Figure 5.3 is a schematic diagram obtained from the Cadence tool that represents a digital circuit design, illustrating various components connected to a central processing unit (CPU).

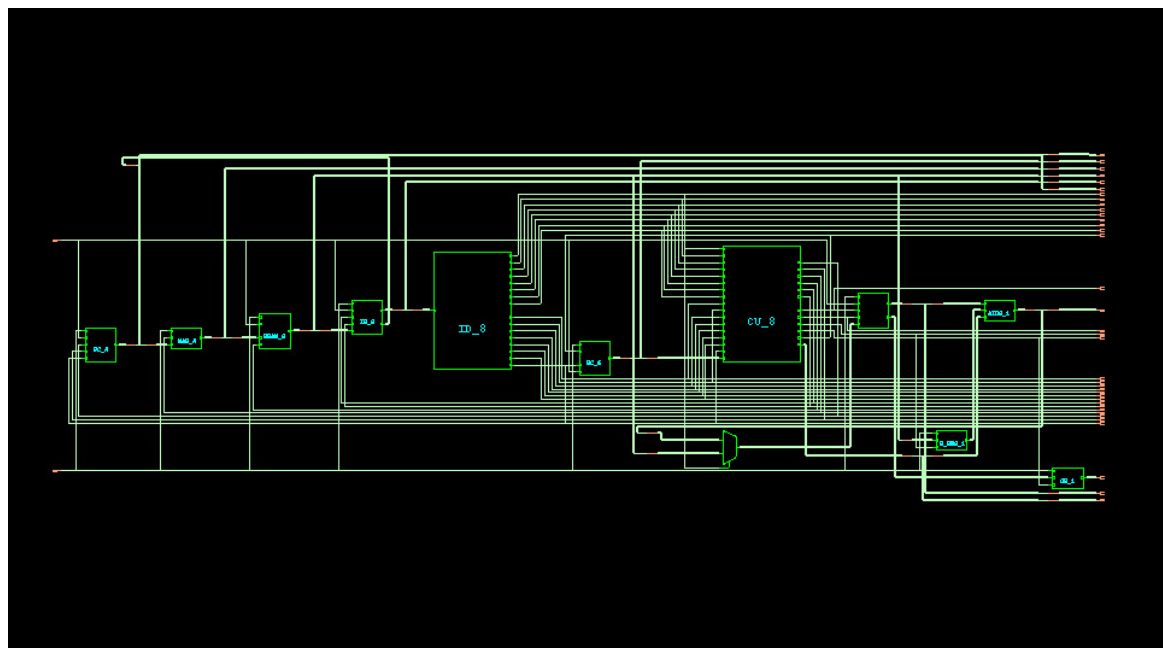


Figure 5.3: - 8-bit Microcontroller Schematic

Figure 5.4 represents a detailed schematic diagram of an 8-bit microcontroller, synthesized using 45 nanometre technology through Cadence Genus tools. This small node size allows for higher transistor density and improved performance while reducing power consumption. It comprises of standard logic gates such as AND, OR, NOT, NAND, and NOR, which are fundamental building blocks. The diagram shows numerous interconnections that represent signal paths, indicating how different components communicate.

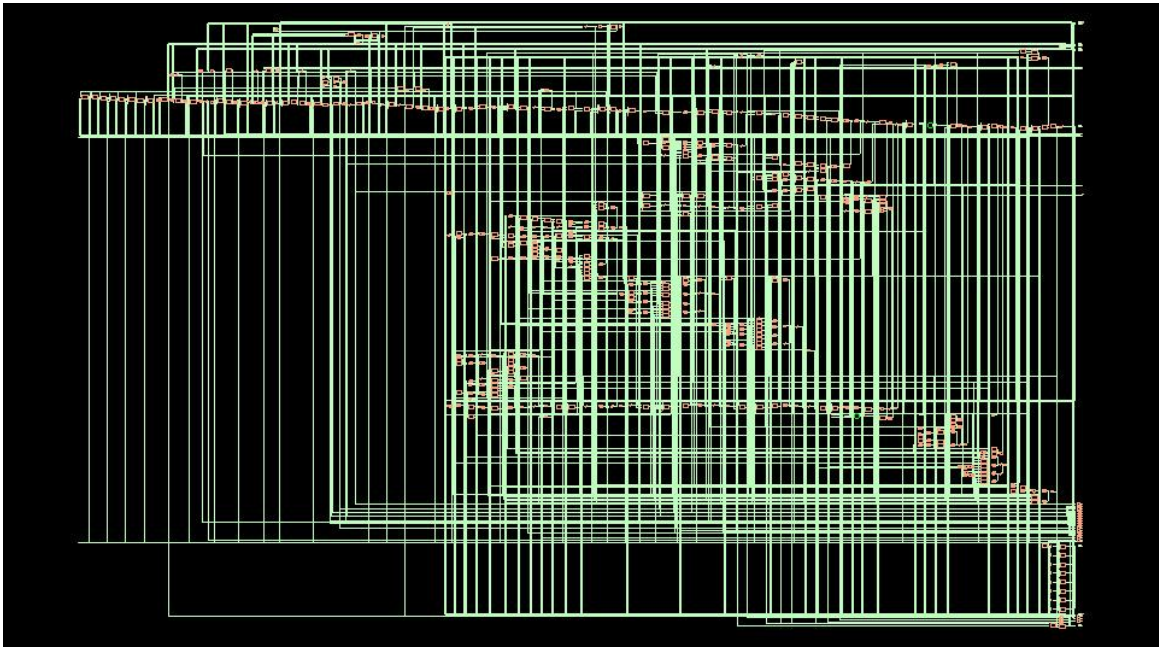


Figure 5.5 presents a detailed breakdown of the power consumption contributions across various categories within the CPU8_1 instance, highlighting the proportions of leakage, internal, and switching power usage. The total power consumption is assessed, with memory and register categories primarily influencing the overall power distribution.

Instance: /CPU8_1
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	2.39957e-08	5.47111e-05	1.62990e-05	7.10341e-05	33.63%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.52491e-07	9.46676e-05	4.25182e-05	1.37338e-04	65.01%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	2.88101e-06	2.88101e-06	1.36%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.76487e-07	1.49379e-04	6.16982e-05	2.11253e-04	100.00%
Percentage	0.08%	70.71%	29.21%	100.00%	100.00%

Figure 5.5: - 8-bit Microcontroller Power reports from Cadence

Figure 5.6 illustrates the power consumption analysis of the 8-bit microcontroller after the implementation of clock gating techniques. This figure provides a comprehensive breakdown of power usage across various components of the microcontroller, emphasizing the impact of clock gating on reducing dynamic power consumption.

Instance: /CPU8_1
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	2.67008e-08	3.54818e-05	1.48871e-05	5.03955e-05	27.08%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.47554e-07	9.34708e-05	3.95948e-05	1.33213e-04	71.59%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	2.77950e-10	6.31218e-07	1.84963e-06	2.48112e-06	1.33%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.74532e-07	1.29584e-04	5.63315e-05	1.86090e-04	100.00%
Percentage	0.09%	69.64%	30.27%	100.00%	100.00%

Figure 5.6: - 8-bit Microcontroller with Clock Gating Power reports from Cadence

Figure 5.7 displays the synthesized schematic of the 8-bit microcontroller incorporating clock gating techniques. This schematic visually represents the microcontroller's architecture, highlighting the integration of clock gating elements designed to optimize power efficiency.

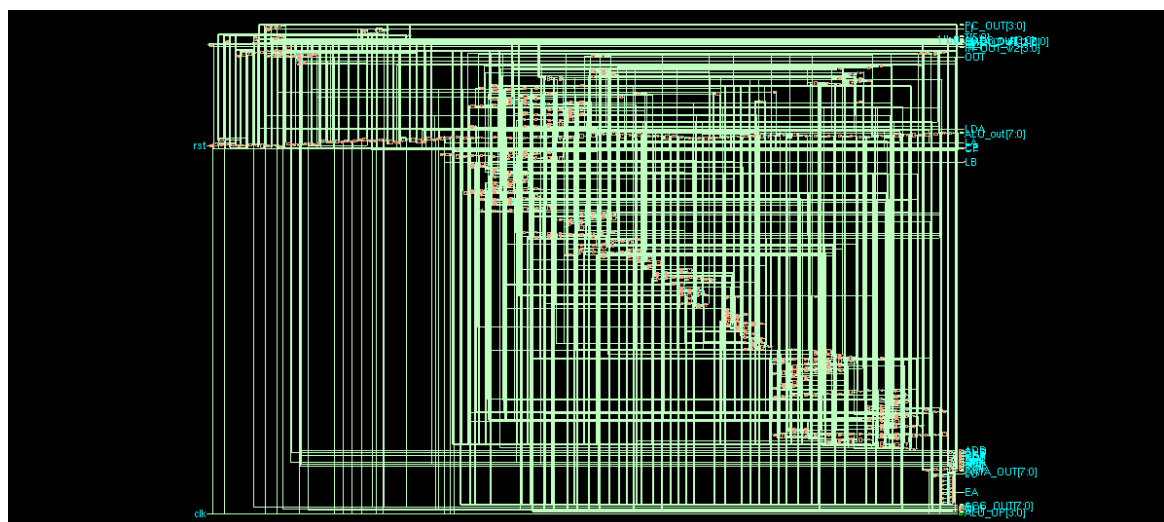


Figure 5.7: - 8-bit Microcontroller with Clock Gating Schematic after synthesis

Figure 5.8 presents the power consumption analysis of the 8-bit microcontroller following the implementation of power gating techniques. This figure provides a detailed breakdown of power usage across various components of the microcontroller, highlighting the significant impact of power gating on reducing both static and dynamic power consumption.

```
Instance: /CPU8_1
Power Unit: W
PDB Frames: /stim#0/frame#0
```

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	2.40077e-08	5.29992e-05	1.32308e-05	6.62540e-05	36.42%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.51311e-07	7.74509e-05	3.51878e-05	1.12790e-04	62.00%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	2.86934e-06	2.86934e-06	1.58%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.75319e-07	1.30450e-04	5.12880e-05	1.81913e-04	100.00%
Percentage	0.10%	71.71%	28.19%	100.00%	100.00%

Figure 5.8: - 8-bit Microcontroller with Power Gating Power reports from Cadence

Figure 5.9 displays the synthesized schematic of the 8-bit microcontroller incorporating power gating techniques. This schematic visually represents the microcontroller's architecture, highlighting the integration of power gating elements designed to optimize power efficiency.

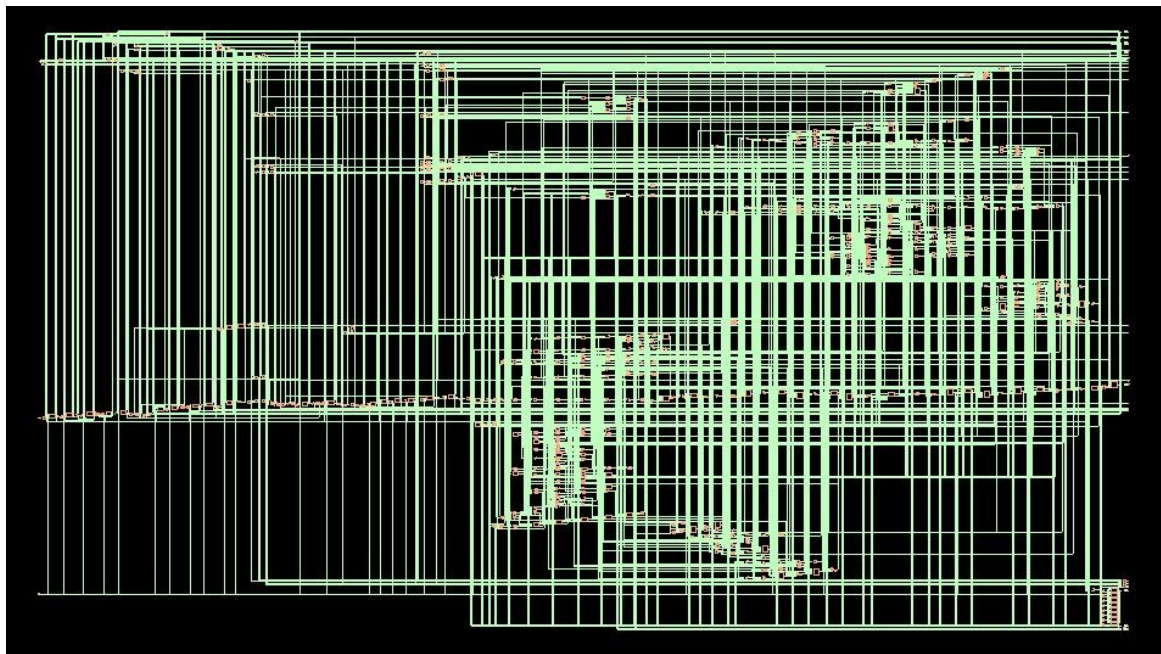


Figure 5.9: 8-bit Microcontroller with Power Gating Schematic after synthesis

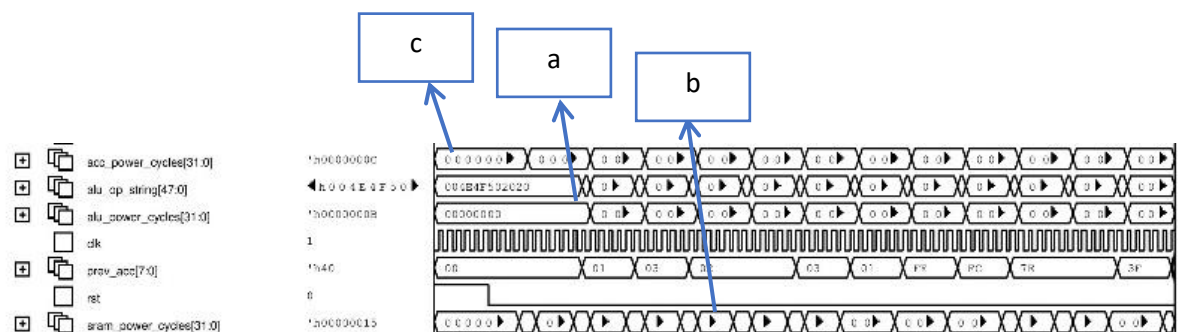


Figure 5.10: - Waveform of the power gated modules

- a) ALU active cycles:11
- b) SRAM active cycles:21
- c) ACC active cycles:12

Figure 5.10 presents the waveform analysis of the power-gated modules within the 8-bit microcontroller. This figure provides a visual representation of the operational behavior of various components as they transition between active and inactive states, showcasing the effectiveness of the power gating technique in managing power consumption.

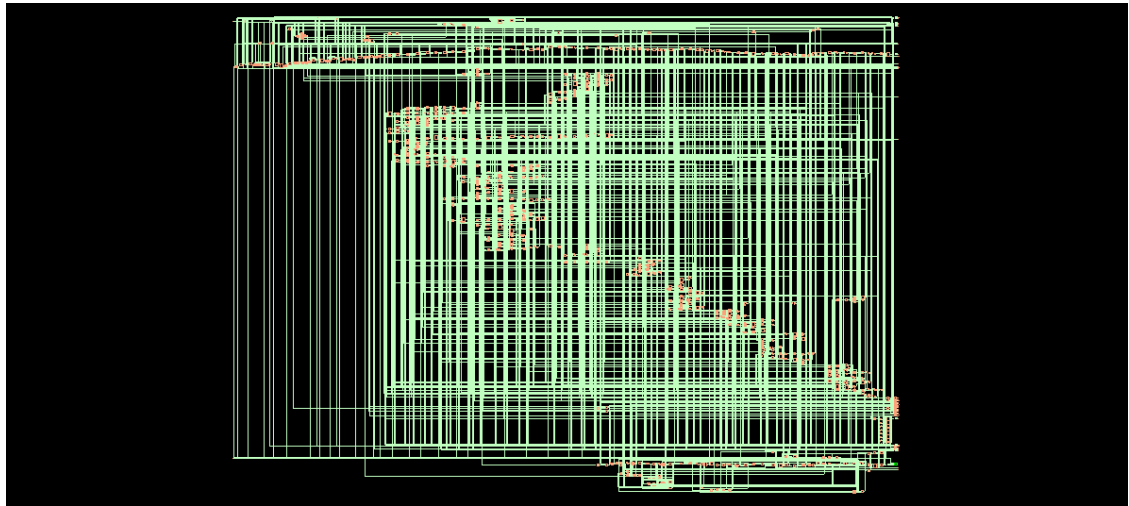


Figure 5.11: 8-bit Microcontroller with Dynamic Voltage Scaling Schematic after synthesis

Figure 5.11 displays the synthesized schematic of the 8-bit microcontroller incorporating dynamic voltage scaling techniques. This schematic visually represents the microcontroller's architecture, highlighting the integration of dynamic voltage scaling elements designed to optimize power efficiency.

Instance: /CPU8_1_DVS
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	3.04279e-08	6.89548e-05	1.87097e-05	8.76950e-05	34.12%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.65200e-07	1.15222e-04	5.04265e-05	1.65814e-04	64.51%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	3.54586e-06	3.54586e-06	1.38%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.95628e-07	1.84177e-04	7.26821e-05	2.57055e-04	100.01%
Percentage	0.08%	71.65%	28.27%	100.00%	100.00%

Figure 5.12: - 8-bit Microcontroller with Dynamic Voltage Scaling Power reports from Cadence

Figure 5.12 presents the power consumption analysis of the 8-bit microcontroller following the implementation of dynamic voltage scaling techniques. This figure provides a detailed breakdown of power usage across various components of the microcontroller, highlighting the significant impact of dynamic voltage scaling on reducing both static and dynamic power consumption.

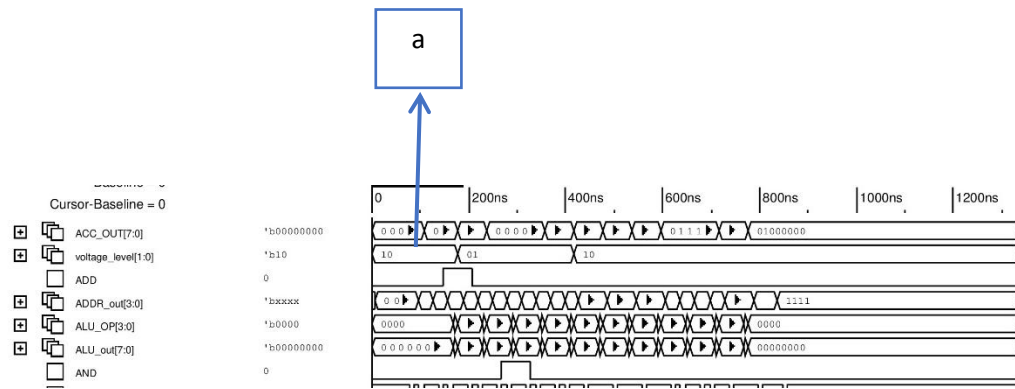


Figure 5.13: Waveform of the Dynamic Voltage Scaling modules

a) The signal represents the voltage level of the operation based on the workload

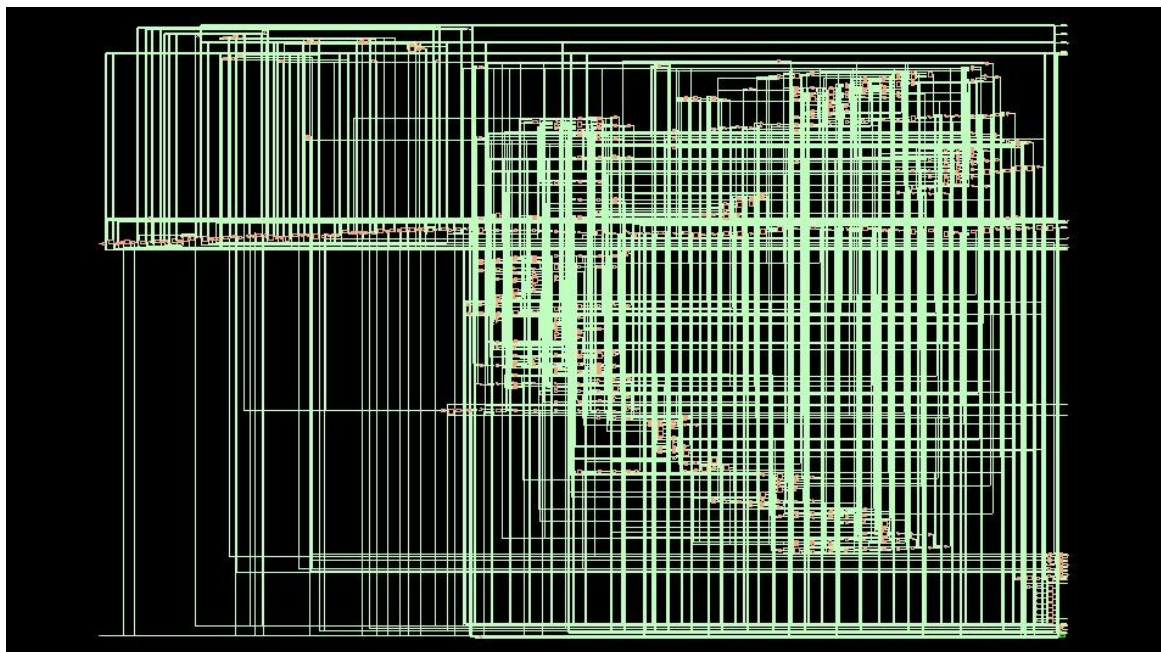


Figure 5.14: 8-bit Microcontroller with Clock gating and Power gating low power techniques applied Schematic after synthesis

Figure 5.14 displays the synthesized schematic of the 8-bit microcontroller following the implementation of two low power techniques applied. This schematic, which includes clock gating and power gating techniques, visually represents the microcontroller's architecture, highlighting the integration of two low power techniques designed to optimize power efficiency.

Instance: /CPU8_1_Combined
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	2.68052e-08	3.62769e-05	1.49661e-05	5.12698e-05	26.96%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.48365e-07	9.44889e-05	4.17989e-05	1.36436e-04	71.73%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	2.77950e-10	6.36921e-07	1.86089e-06	2.49809e-06	1.31%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.75448e-07	1.31403e-04	5.86259e-05	1.90204e-04	100.00%
Percentage	0.09%	69.09%	30.82%	100.00%	100.00%

Figure 5.15: - 8-bit Microcontroller with Clock gating and Power gating low power techniques applied Power reports from Cadence

Figure 5.15 presents the power consumption analysis of the 8-bit microcontroller following the implementation of Clock gating and Power gating low power techniques. This figure provides a detailed breakdown of power usage across various components of the microcontroller, highlighting the significant impact of all three low power techniques on reducing both static and dynamic power consumption.

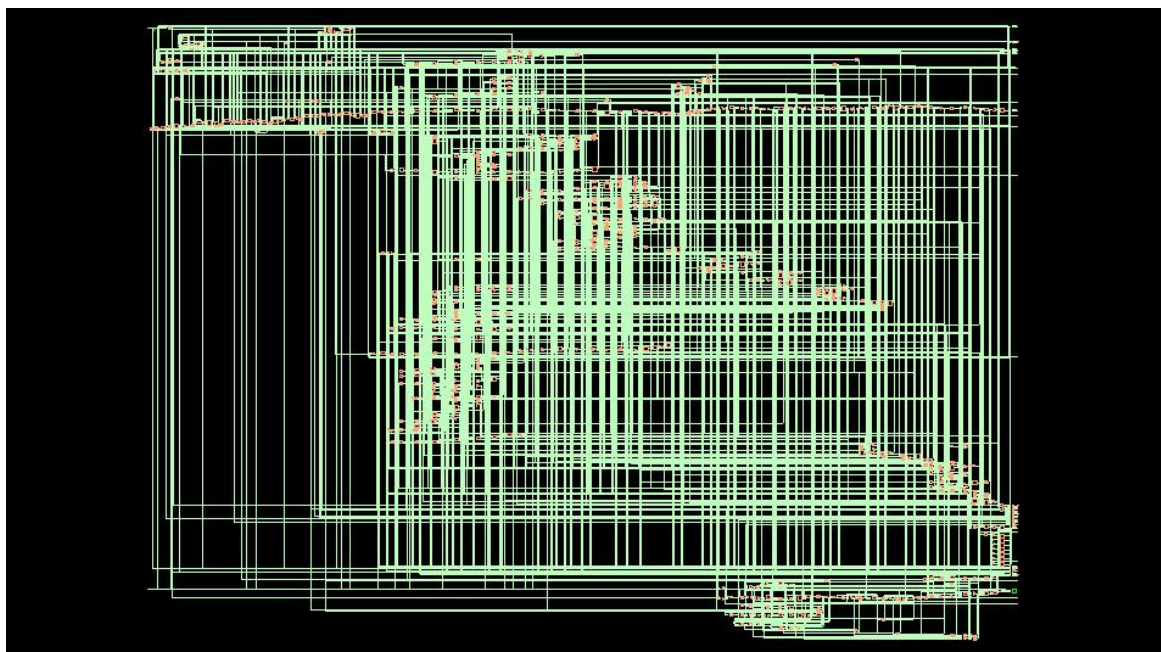


Figure 5.16: 8-bit Microcontroller with all three low power techniques applied Schematic after synthesis

Figure 5.16 displays the synthesized schematic of the 8-bit microcontroller following the implementation of all three low power techniques applied. This schematic visually represents the microcontroller's architecture, highlighting the integration of all three low power techniques designed to optimize power efficiency.

Instance: /CPU8_1_Combined
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	3.29103e-08	5.09594e-05	1.75293e-05	6.85216e-05	30.37%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.68172e-07	1.06409e-04	4.73532e-05	1.53930e-04	68.22%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	2.77950e-10	6.37058e-07	2.53417e-06	3.17150e-06	1.41%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	2.01360e-07	1.58005e-04	6.74167e-05	2.25623e-04	100.00%
Percentage	0.09%	70.03%	29.88%	100.00%	100.00%

Figure 5.17: - 8-bit Microcontroller all three low power techniques applied Power reports from Cadence

Figure 5.17 presents the power consumption analysis of the 8-bit microcontroller following the implementation of all three low power techniques. This figure provides a detailed breakdown of power usage across various components of the microcontroller, highlighting the significant impact of all three low power techniques on reducing both static and dynamic power consumption.

Table 5.1: - Comparison Table of the 8 bit RISC Microcontroller Designs

Design Configuration	Leakage Power (W)	Internal Power (W)	Switching Power (W)	Total Power (W)	Percentage Increased
Baseline (No Techniques)	1.76487e-07	1.49379e-04	6.16982e-05	2.11253e-04	
Clock Gating	1.74532e-07	1.29584e-04	5.63315e-05	1.86090e-04	11.9%
Power Gating	1.75319e-07	1.30450e-04	5.12880e-05	1.81913e-04	13.9%
Clock gating and Power gating Combined	1.75448e-07	1.31403e-04	5.86259e-05	1.90204e-04	9.97%

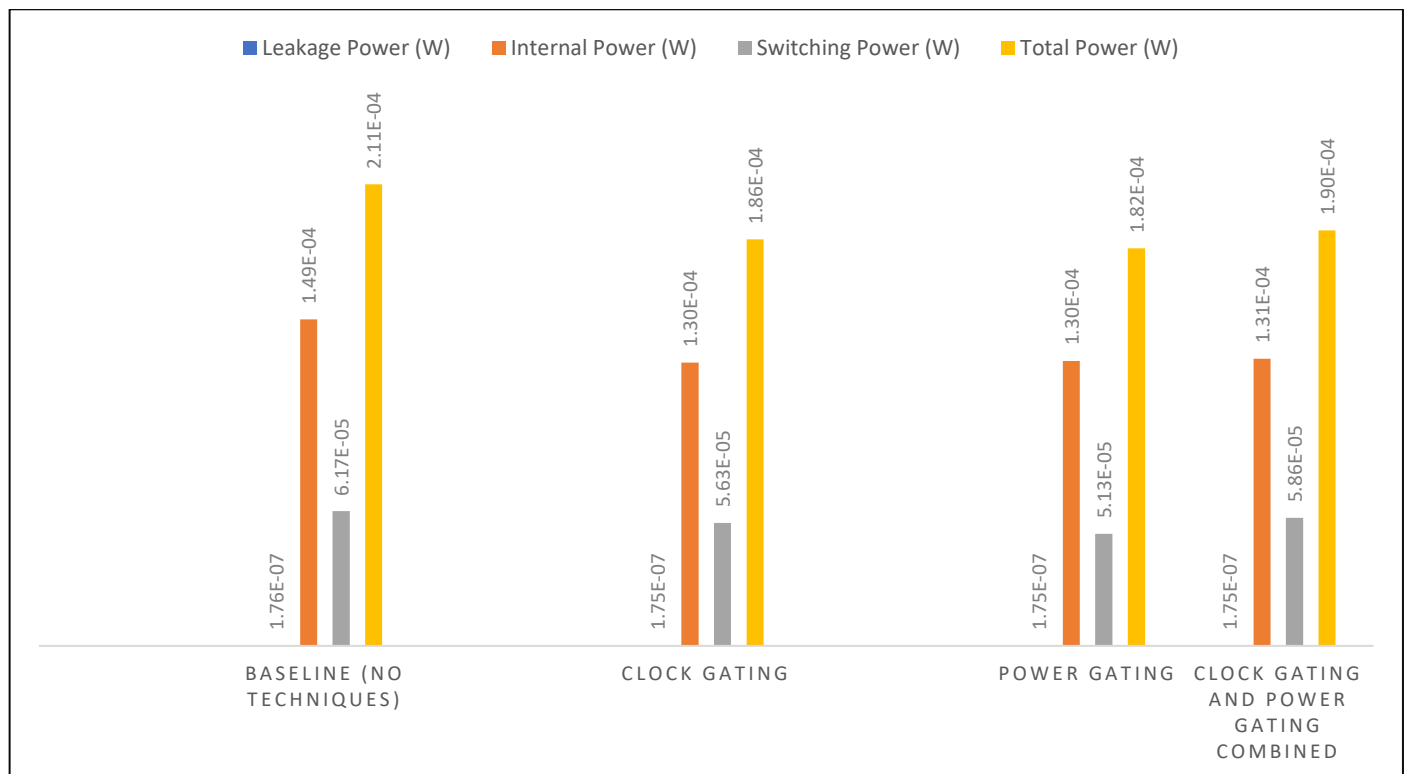


Figure 5.18: - Comparison of Power Consumption Across Different Low-Power Techniques

From the observation of table 5.1 and Figure 5.18 we can say that:

1. Total Power Consumption:

- **Lowest:** Power Gating – $1.81913e-04$ W
- **Highest:** Baseline (No Techniques) – $2.11253e-04$ W
- **Clock Gating:** $1.86090e-04$ W – shows a significant reduction compared to the baseline.
- **Combined Clock and Power Gating:** $1.90204e-04$ W – results in a reduction, though not as effective as individual techniques.

2. Effectiveness of Power Optimization Techniques:

- **Clock Gating:**
 - Reduces total power by approximately 11.9% compared to the baseline (from $2.11253e-04$ W to $1.86090e-04$ W).
 - Major savings are observed in internal and switching power.
- **Power Gating:**
 - Most effective technique, achieving a 13.9% reduction in total power (to $1.81913e-04$ W).
 - Notably lowers switching power, making it the most energy-efficient among all configurations.
- **Combined Clock and Power Gating:**
 - Yields only a ~9.97% reduction in total power (to $1.90204e-04$ W), which is less than either technique individually.

CHAPTER 6

CONCLUSION

Chapter 6

CONCLUSION

6.1 Conclusion

The development of a low-power 8-bit Reduced Instruction Set Computer (RISC) microcontroller addresses the critical challenge of energy efficiency in modern embedded systems and portable electronics.

Traditional microcontroller designs often fail to meet the stringent power consumption requirements of contemporary applications, resulting in diminished battery life and increased operational costs. This project specifically aimed to create a microcontroller that not only meets these demands but also enhances performance through innovative design strategies.

Key approaches employed in this design included:

1. **RISC Architecture:** The choice of RISC architecture facilitated a streamlined instruction set and reduced control logic, allowing for rapid instruction execution with minimal cycles. This design choice is particularly beneficial for applications in the Internet of Things (IoT) and wearable technologies, where efficiency is paramount.

2. **Power Management Techniques:** Three primary strategies were integrated into the design:

- **Clock Gating:** This technique selectively disables clock signals to inactive components, resulting in a power reduction of approximately 11.9% compared to the baseline design, achieving a total power consumption of $1.86090\text{e-}04$ W.
- **Power Gating:** By completely shutting off power to inactive units, power gating proved to be the most effective technique, yielding a 13.9% reduction in total power consumption, bringing it down to $1.81913\text{e-}04$ W.
- **Dynamic Voltage Scaling (DVS):** Although intended to optimize power based on workload demands, DVS resulted in an increase in total power consumption by 21.7%, reaching $2.57055\text{e-}04$ W. This outcome highlights the need for careful management of voltage and frequency adjustments to avoid performance degradation.

3. Combined Techniques: The integration of the two techniques that led to a slight reduction of 9.96% compared to the baseline, with a total power consumption of 1.90204e-04 W. This indicates that while combining techniques can yield benefits, their effectiveness may vary based on workload and operational conditions.

The simulation results validated the functionality and performance of the microcontroller, confirming that the design meets the necessary criteria for low-power operation. The successful implementation of critical components, including the Arithmetic Logic Unit (ALU), Control Unit (CU), and Program Counter (PC), was validated through detailed waveform outputs, ensuring efficient operation across various states of instruction processing.

Looking ahead, the scalability of this microcontroller design presents opportunities for enhancements, such as the development of 16-bit or 32-bit versions to meet evolving technological demands. Additionally, further research could explore the integration of machine learning algorithms for adaptive power management, potentially leading to even greater energy efficiency.

In summary, the work accomplished in this project not only addresses current challenges in low-power computing but also lays the foundation for future advancements in microcontroller technology. It contributes to the growing trend toward energy-efficient solutions in contemporary electronics, emphasizing the importance of innovative design strategies in developing sustainable microprocessors that meet the needs of modern applications.

REFERENCES

1. Gaonkar, R. (2013). Microprocessor architecture, programming and applications with the 8085 (6th ed.). Penram International Publishing.
2. Mano, M. M. (2007). Computer system architecture (3rd ed.). Pearson Education India.
3. Bhardwaj, P., & Murugesan, S. (2016). Design & simulation of a 32-bit RISC based MIPS processor using Verilog. *International Journal of Research in Engineering and Technology*, 5, 1-6.
4. Kaur, R., & Anuj, A. (2014). 8 bit RISC processor using Verilog HDL. *International Journal of Engineering Research and Applications*, 4(3), 417-422.
5. Uma, R. (2012). Design and performance analysis of 8-bit RISC processor using Xilinx tools. *International Journal of Engineering Research and Applications*, 2, 053-058.
6. Kumar, B. R., Ravisaketh, & Kumar, S. (2014). Implementation of a 16-bit RISC processor for convolution application. *Research India Publications*, 441-446.
7. Mano, M. M. (2012). Digital design (5th ed.). Prentice Hall.
8. Begum, M., & Kumar, M. (2012). FPGA based implementation of 32-bit RISC processor. *International Journal of Engineering Research and Applications*, 1, 1148-1151.
9. Nieto, O. C., & Huerta Ruelas, J. A. (2015). Design of a general purpose 8-bit RISC processor for computer architecture learning. *Computers & Systems*, 19(2), 371-385.
10. Oztekin, H., Temurtas, F., & Gulbag, A. (2011). Microprocessor architecture design on reconfigureurable hardware as an educational tool. *IEEE Computer Society*, 385-389.
11. Lee, I., Lee, D., & Choi, K. (2008). ODALRISC: A small, low power, and configureurable 32-bit RISC processor. In *International System on Chip Design Conference*.
12. Ritpurka, S. P., Thakare, M. N., & Korde, G. D. (2014). Review on 32-bit MIPS RISC processor using VHDL. In *International Conference on Advances in Engineering and Technology* (pp. 46-50).
13. Topiwala, M. N., & Sarawathi, N. (2013). Implementation of a 32-bit MIPS based processor using FPGA. In *Proceedings of the National Conference on Recent Advances in Electronics and Computer Engineering*.

14. Chintaiah, N., & Reddy, G. U. (2018). FPGA implementation of coding minimizing delay and power in SoC interconnects. In Springer International Conference on Soft Computing and Signal Processing (pp. 427-435).
15. Musoll, E., Lang, T., & Cortadella, J. (2017). Exploiting the locality of memory references to reduce the address bus energy. In Proceedings of the ACM/IEEE International Symposium on Low Power Electronics Design (pp. 202-207).
16. Park, A., & Farrens, M. (2019). Address compression through base register caching. In Annual ACM/IEEE International Symposium on Microarchitecture (pp. 193-199).
17. Benini, L., De Micheli, G., Macii, E., Sciuto, D., & Silvano, C. (2017). Asymptotic zero-transition activity encoding for address buses in low-power microprocessor-based systems. In IEEE/ACM Great Lakes Symposium on VLSI (pp. 77-82).
18. Aghaghiri, Y., Fallah, F., & Pedram, M. (2016). ALBORZ: Address level bus power optimization. In International Symposium on Quality Electronic Design (pp. 470-475).
19. Sotiriadis, P., & Chandrakasan, A. (2000). Low power bus coding techniques considering inter-wire capacitances. In Custom Integrated Circuits Conference (pp. 507-510).
20. Macchiarulo, L., Macii, E., & Poncino, M. (2001). Low-energy for deep-submicron address buses. In International Symposium on Low Power Electronics Design (pp. 176-181).
21. Benini, L., Macii, A., Poncino, M., & Scarsi, R. (2000). Architectures and synthesis algorithms for power-efficient bus interfaces. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 19(9), 969-980.
22. Stan, M. R., & Burleson, W. P. (2015). Bus-invert coding for low-power I/O. *IEEE Transactions on Very Large-Scale Integration (VLSI) Systems*, 23(1), 49-58.
23. Zhang, Y., Lach, J., Skadron, K., & Stan, M. R. (2002). Odd/even bus invert with two-phase transfer for buses with coupling. In International Symposium on Low Power Electronics Design (pp. 80-83).
24. Austin, M. T. (2004). Low power techniques for microprocessor design. In IEEE Symposium on VLSI Circuits Digest of Technical Papers (pp. 114-119).
25. Smith, A. M., & Adams, D. J. (2003). Energy-aware compilation and execution in Java-enabled mobile devices. In International Conference on Parallel and Distributed Processing Symposium (pp. 34a).

Annexure 1

**Review Paper presented in the International Conference on
Sustainable Computing Intelligent Systems (ICSCIS-2025),
Poornima University, Jaipur.**

Annexure 2

Paper submitted in the IEEE INDISCON-2025, NIT Rourkela

Annexure 3

Tools Used

Genus by Cadence:

Cadence Genus is a comprehensive tool from Cadence Design Systems that focuses on synthesis and design implementation for digital circuits, particularly for Application-Specific Integrated Circuits (ASICs). Genus is designed to optimize the synthesis process, enabling designers to achieve high-performance, low-power, and area-efficient designs. Genus employs advanced synthesis algorithms and optimization techniques to translate high-level Hardware Description Language (HDL) code, such as Verilog and VHDL, into an optimized gate-level netlist. The tool supports multi-objective optimization, allowing designers to balance various design constraints, including area, power, and performance.

The manual for Cadence Genus can be found at the following web address:

- https://www.cadence.com/content/dam/cadence-www/global/en_US/documents/tools/digital-design-and-signoff/genus-synthesis-solution/genus-user-guide.pdf

To run the synthesis in a cadence tool use the commands below in the terminal :

- Open a folder and upload the Verilog files and open terminal
- In terminal type, `csh`
- `source /home/install/cshrc`
- `xrun file.v file_tb.v -access +rwc -gui`
- `read_hdl /home/cadence/Desktop/ujwal_8thsemproject/rcfiles/rtl/CPU8_1.v`
- `set_attribute library /home/cadence/Desktop/ujwal_8thsemproject/rcfiles/lib/slow.lib`
- `set_attribute lib_search_path /home/cadence/Desktop/ujwal_8thsemproject/rcfiles/rtl/`
- `elaborate`
- `create_clock -name clk -period 10 [get_ports clk]`
- `set_dont_touch CPU8_1/clk_buf`
- `syn_generic`
- `syn_map`
- `syn_opt`
- `report power > power`

Isim by Xilinx ISE 14.2:

The Xilinx ISE Simulator-isim provides a complete, full-featured HDL simulator integrated within ISE. HDL simulation now can be an even more fundamental step within your design flow with the tight integration of the isim within your design environment. The Xilinx ISE Simulator (isim) is a Hardware Description Language (HDL) simulator that enables you to perform functional and timing simulations for VHDL, Verilog and mixed language designs. The key elements comprising the ISE Simulator environment are Vhpcomp (VHDL compiler), fuse (HDL elaborator and linker), Simulation Executable and isimgui (isim Graphical User Interface).

The manual for Isim can be found in the following web address:

- http://www.xilinx.com/support/documentation/sw_manuals/xilinx11/ug682.pdf